ECDSA.Fail: Open Autoresearch for Optimizing Elliptic-Curve Point Addition in Shor's Algorithm

Jieyi Long[1]✉, Theodore Pender[2], Zhao Huang[3], Manuel B. Santos[4], Samrendra Kumar Singh[5], Bartosz Naskręcki[6,7], BitWonka[8], Joe Doyle[9], Pierre-Luc Dallaire-Demers[10], Francesco Giannicola, Ruben M. L. Paschoarelli[4], Oli Freuler[5], Jackie Chia-Hsun Lee[11], Vasily Gnuchev, Gopi Kannappan, John Boyer, Xavier Butler, Akash Balasubramani[5], Jordan Newman, Bereket Dereje, Alexander Hertlein[12], Robert Kodra[2], Lucas Levy[5], Shaan Patel, JT Rose, Matt Zweil, Okechukwu Wisdom[13], Tarek El-Eter[2], Edison Lee, Michael Dong[3], Alan Li[3], Anto Joseph[14], Other Leaderboard Contributors*, Gajesh Naik[14]✉, Gautham Anant[14]✉, Soubhik Deb[14]✉, Justin Drake[15]✉

[1]Theta Labs [2]Starknet Foundation [3]Brevis [4]MultiVM Labs [5]StarkWare [6]Adam Mickiewicz University Poznań [7]Warsaw University of Technology [8]Octav [9]Trail of Bits [10]Pauli Group [11]ScienceVR [12]Sei Labs [13]Stanford Free Systems Lab [14]Eigen Labs [15]Ethereum Foundation

Abstract. We propose *Open Autoresearch*, a paradigm in which humans and AI agents address optimization problems by publishing evaluator-verified improvements to a public leaderboard. We instantiate it in ECDSA.Fail[1], whose participants optimized a reversible `secp256k1` point-addition circuit, a bottleneck in Shor's algorithm for elliptic-curve cryptography. The benchmark minimizes the spacetime-inspired score $S = Q \times T$, where Q is peak logical qubit width and T is average executed Toffoli count. Participants reduced S by 86.1%, with the best circuit at the data cutoff (26 July 2026) reaching $Q = 1{,}151$ and $T = 1{,}299{,}453$, giving $Q \times T \approx 1.496$B. This is more than 50% below the reported score of Google's point-addition circuit (un)disclosed via a zero-knowledge proof [3]. Because the benchmark supplies one addend classically, we construct a coherent windowed-addition-compatible variant implementing the single-call interface required by windowed Shor. It uses $Q = 1{,}162$ and $T = 1{,}684{,}161$; on 100,000 random inputs, its empirical success probability was $\hat{p} = 0.99809$, corresponding under an independently rerunnable per-call sensitivity model to the proxy $Q \times T/\hat{p} \approx 1.961$B. Its Q and T lie below the Google and Schrottenloher operating points [3, 59], although differing interfaces and accounting conventions preclude formal dominance. After the cutoff, the $Q \times T$ score was further reduced to 1.259B, while a separate low-width circuit reached 813 qubits. The public record shows that AI agents complemented human judgment, providing evidence for open autoresearch on efficiently evaluable, machine-checkable objectives.

* See Appendix A for the complete contributor list.

[1] Project website and the latest results: `https://ecdsa.fail`; source repository: `https://github.com/Layr-Labs/ecdsafail-challenge`.

Table of Contents

1 Introduction . . . 3
1.1 Main Contributions . . . 5
1.2 Related Work . . . 6
1.3 Organization of the Paper . . . 7
2 Preliminaries . . . 8
2.1 Quantum Computing Background . . . 8
2.2 Elliptic Curve Cryptography and Quantum Attacks . . . 10
3 The ECDSA.Fail Challenge and the Benchmark . . . 16
3.1 The Point-Addition Task . . . 16
3.2 Circuit Model and Cost Metric . . . 17
3.3 Correctness, Verification, and Scoring . . . 18
3.4 Formulating the Circuit Optimization Problem . . . 19
4 The Open Autoresearch Paradigm . . . 20
4.1 Overview of Open Autoresearch . . . 20
4.2 The ECDSA.Fail Open Autoresearch Harness . . . 21
4.3 ECDSA.Fail Autoresearch Techniques . . . 22
4.3.1 Overview of the Autoresearch Scaffold and Methodology . . . 22
4.3.2 Observed Autoresearch Practices . . . 23
4.3.3 Why It Works and What We Have Learned . . . 25
4.4 Consolidated ECDSA.Fail Autoresearch Framework . . . 26
5 Results and Findings . . . 28
5.1 Overall Summary of Results . . . 28
5.1.1 The Baseline and the Pareto Frontier at the Data Cutoff . . . 28
5.1.2 Comparison with Prior Point-Addition Results . . . 30
5.2 Best ECDSA.Fail Point-Addition Circuits at the Data Cutoff . . . 30
5.2.1 Best $Q \times T$-Scoring Circuit . . . 32
5.2.2 Best low-Q Circuit . . . 33
5.2.3 Windowed-Addition Compatibility of the Best-Scoring Circuit . . . 35
5.3 Principal Circuit Optimization Techniques . . . 37
5.3.1 The Jump-2 Euclidean Algorithm . . . 37
5.3.2 Base-5 Codec for the Jump-2 Transcript . . . 39
5.3.3 Comparison-Free Ping-Pong Dialog-GCD . . . 40
5.3.4 Karatsuba Squaring with Pseudo-Mersenne Modulo . . . 42
5.3.5 Register-Shared EEA and Lane-Lifetime Refinements . . . 43
5.3.6 Constant Propagation . . . 46
5.3.7 Dead Code Elimination . . . 46
5.4 Evolution of the ECDSA.Fail Point-Addition Circuit . . . 47
5.4.1 Raw and Retry-Adjusted $Q \times T$ Scores Across Submissions . . . 47
5.4.2 Three Eras of Circuit Optimization for the $Q \times T$ Score . . . 49
5.4.3 Low-Qubit and Pareto Exploration . . . 50
5.4.4 Breakdown of Optimization Techniques Across the Circuit Optimization Trajectory . . . 51
5.5 Reproducibility of the Results . . . 53
6 Limitations, Safety, and Responsible Framing . . . 54
7 Conclusion and Outlook . . . 56
A Contributor Statement . . . 61

1 Introduction

Quantum computers exploit superposition, entanglement, and interference to obtain asymptotic advantages for certain computational problems [47]. That same capability threatens widely deployed public-key cryptography [61]. A *cryptographically relevant quantum computer* (CRQC) could execute Shor's algorithm [61] at cryptographic scale, defeating RSA through integer factorization and elliptic-curve cryptography through the elliptic-curve discrete logarithm problem (ECDLP) [52]. We refer to the arrival of such a machine as *Q-day*. Although its timing remains uncertain, migration away from vulnerable cryptography is already under way. NIST has standardized post-quantum replacements, and the initial public draft of NIST IR 8547 proposes deprecating classical public-key algorithms at the 112-bit security level after 2030 and disallowing them after 2035 [44, 45]. U.S. federal guidance further calls for migrating digital signatures in sensitive federal systems by 2031 [49]. The threat is particularly consequential for `secp256k1`, used by Bitcoin and Ethereum for transaction authorization [10, 28, 43, 66]: a CRQC capable of solving ECDLP could recover private signing keys from exposed public keys and forge signatures [51, pp. 40–41].

When will Q-day come? The answer depends partly on the logical resources required by Shor's ECDLP circuit. Beginning with Proos and Zalka's detailed prime-field construction and Kaye and Zalka's space-efficient reversible techniques [31, 52], successive work has reduced these estimates through explicit Toffoli circuits, windowed arithmetic, and architecture-specific implementations [25, 27, 36, 54]. Google Quantum AI recently reported a further two- to threefold reduction over prior published estimates, attested by a zero-knowledge proof without releasing the circuit; Schrottenloher subsequently obtained comparable figures with an open, fully specified design [3, 59]. At the physical layer, Cain et al. estimate that a reconfigurable neutral-atom architecture using high-rate qLDPC codes could solve a P-256 discrete logarithm with as few as 10,000 physical qubits, or within days using roughly 26,000, since additional qubits buy parallelism [9]. Under explicit hardware and error-correction assumptions, Dallaire-Demers et al. place a full 256-bit ECDLP instance within a 2027–2033 window [14].

Traditionally, similar to other scientific research domains, leading quantum-circuit constructions have been developed by individual specialists or small teams through manual design and heuristic search [3, 27, 36, 39, 54, 59]. In recent years, LLM-based agents, including general-purpose coding agents such as Claude Code and Codex, have begun to automate parts of scientific research and broaden access to specialized computational workflows [2, 16, 38, 48]. Such systems can synthesize research ideas from the literature, modify and execute code, and iteratively improve solutions against quantitative evaluators with limited supervision [2, 38, 48]. Their outputs nevertheless remain hypotheses until checked. With a machine-checkable evaluator, the practical bottleneck can shift from manual implementation toward hypothesis generation and verification.

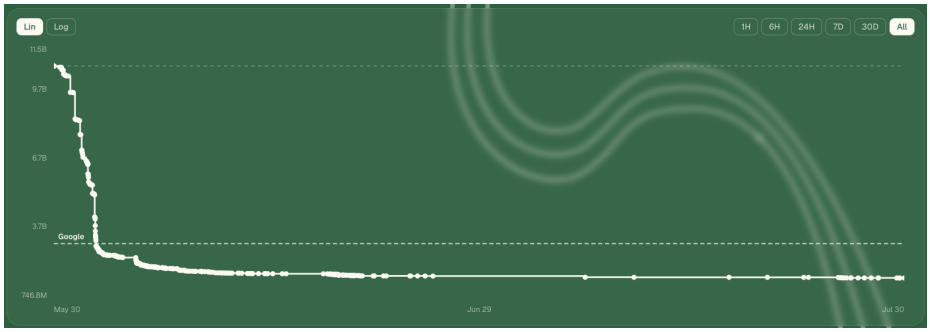


Fig. 1: Evolution of the best ECDSA.Fail $Q \times T$ score, from the 10.75B baseline to 1.496B at the data cutoff, an 86.1% reduction. The dashed line marks the reported Google/Babbush et al. [3] operating point of 2.993B. The challenge score is more than 50% lower, although differing interfaces and accounting conventions make the comparison contextual.

Building on these capabilities, we introduce *Open Autoresearch*: a hard problem is expressed as a measurable objective with a machine-checkable evaluator, and a heterogeneous, largely self-organizing community of humans and AI agents competes and builds on shared results through a public leaderboard. Although developed here for quantum-circuit optimization, the paradigm may apply to other problems with efficiently verifiable objectives. It extends recent AI-driven approaches to program search, evolutionary coding, and automated experimentation [19, 38, 41, 48, 55], while emphasizing two features. First, it is *open*: independent participants share a target and public artifacts, including documented failures, without requiring prior coordination. Second, it is *observable*: progress forms a public sequence of verified, attributable steps, making the research process itself available for analysis. We hypothesize that this combination can help address frontier optimization problems with machine-checkable outcomes.

Our evidence comes from ECDSA.Fail, an open competition in which more than one hundred participants and their agents optimized a reversible `secp256k1` mixed point-addition circuit, minimizing $Q \times T$, where Q is peak logical width and T is average executed Toffoli count. As Figure 1 shows, submissions reduced the score from the 10.75B baseline ($Q = 2{,}715$, $T = 3{,}960{,}753$) to 1.496B ($Q = 1{,}151$, $T = 1{,}299{,}453$) at the data cutoff (26 July 2026, 09:21:55 UTC), an 86.1% reduction. In Figure 2, this circuit lies below the reported Google/Babbush et al. [3] and Schrottenloher [59] operating points on both axes, with a product more than 50% below Google's. To our knowledge, it had the lowest reported numerical $Q \times T$ among `secp256k1` point-addition circuits as of the data cutoff. These comparisons are contextual rather than claims of formal dominance because interfaces, correctness assumptions, and accounting conventions differ. In particular, the benchmark supplies one addend classically, whereas windowed Shor selects it coherently. We therefore construct a windowed-addition-compatible single-call variant using 1,162 qubits and approximately 1.68 million

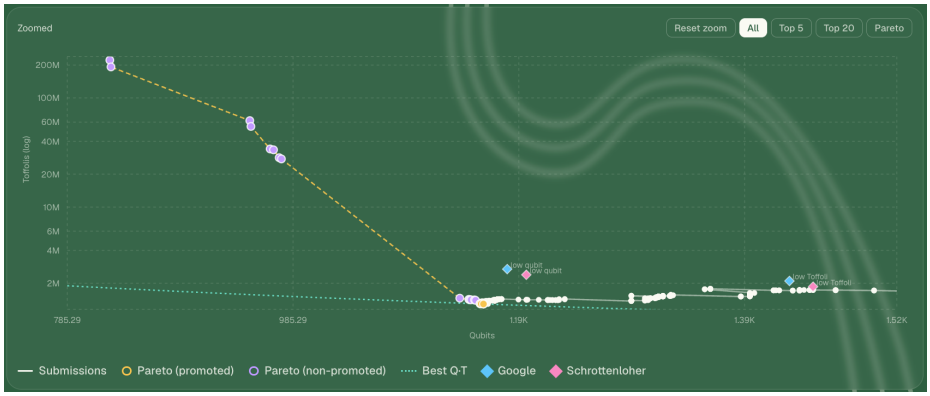


Fig. 2: ECDSA.Fail qubit–Toffoli Pareto frontier. White points are challenge submissions; orange and purple circles are promoted and non-promoted admitted Pareto points; the cyan dotted curve marks the best observed $Q \times T$; and diamonds mark the Google/Babbush [3] (blue) and Schrottenloher [59] (pink) operating points. External points are contextual because interfaces and accounting conventions differ; see Tables 2 and 3.

average executed Toffoli gates, also below Google's and Schrottenloher's reported operating points [3,59]. Like prior constructions [3,59], our circuits trade a small error rate for lower resources. On a longitudinal sample evaluated over 50,000 pseudorandom inputs, the retry-adjusted score $Q \times T/\hat{p}$ closely tracks the raw score because the empirical success rate $\hat{p}$ remains near one (Section 5.4.1). The score omits Toffoli depth and parallelism, so circuits with identical Q and T score equally even when their schedules differ (Section 6).

1.1 Main Contributions

Our contributions span the research methodology, optimized circuits, and public record of their development.

- **The Open Autoresearch paradigm.** We formulate open autoresearch as a verifier-gated paradigm in which anyone may participate without formal entry requirements, and humans and AI agents optimize a shared objective through a public evaluator, repository, leaderboard, and experimental record. Verified improvements become foundations for further work, while documented failures provide reusable evidence. To the best of our knowledge, ECDSA.Fail is the first ever large-scale open-autoresearch project of this kind, with more than 100 leaderboard contributors and over 400 promoted submissions at the data cutoff. From the practices observed during ECDSA.Fail, we derive a framework combining parallel search, human steering, verification, shared memory, and evolving agent skills (Section 4). Participant-contributed harnesses and scaffolds are available in a public repository.

- **Frontier-level point-addition circuits.** The ECDSA.Fail challenge optimizes a reversible `secp256k1` mixed point-addition circuit. At the data cutoff, the best circuit has a $Q \times T$ product more than 50% below the reported Google/Babbush et al. point, with reported Q and T below both the Google/Babbush et al. and Schrottenloher constructions [3, 59] (Section 5.1). The improvement remains substantial under the retry-adjusted proxy $Q \times T/\hat{p}$ (Section 5.4.1). Because accounting conventions differ, these comparisons are contextual rather than claims of formal dominance. Additional operating points characterize the Pareto frontier and low-width regime:
  - The best $Q \times T$ submission at the data cutoff, `8e9c9a2`, uses $Q = 1{,}151$ and $T = 1{,}299{,}453$ (Section 5.2.1) and achieved a score of 1.496B; its arithmetic and principal optimizations are detailed in Supplemental Note D. Based on this circuit, we additionally construct a coherent windowed-addition-compatible variant with $Q = 1{,}162$ and $T = 1{,}684{,}161$ (Section 5.2.3), documented in Supplemental Note F. After the data cutoff, submission `3616dbf` introduced a novel comparison-free "ping-pong" GCD technique (Section 5.3.3), which further reduced the benchmark score to approximately 1.259B, using $Q = 1{,}321$ and $T = 952{,}707$. This was the first result reported with fewer than one million average executed Toffoli gates and represents a 15.9% reduction in $Q \times T$ relative to the cutoff incumbent.
  - At the cutoff, the best low-Q submission, `b6f2b0a`, used 825 qubits (Section 5.2.2; Supplemental Note E). A later submission, `b4a51d2`, reported 813 qubits, 22 below Luo et al.'s recent 835-qubit result [39].
- **Source-auditable architectures and optimizations.** We provide reproducible implementations and technical descriptions of the resulting circuits. They build on the record-and-replay Euclidean architecture (a.k.a dialog-GCD) originally introduced by Khattar et al. [33], together with the TrailMix implementation substrate [64]. Principal optimizations include jump-2 Euclidean steps, compressed transcript codecs, ping-pong dialog-GCD, Karatsuba squaring with pseudo-Mersenne modulo, register-shared extended Euclidean inversion, constant propagation, and dead-code elimination (Sections 5.3.1 to 5.3.7). A complete catalog appears in Supplemental Note B.
- **Longitudinal and reproducible evidence.** We reconstruct the improvement trajectory, Pareto frontier, low-width branches, and three successive resource regimes, and classify 400 scored source commits by their primary optimization mechanism (Section 5.4). We release circuit sources, submission histories, frozen leaderboard data, operating-point manifests, and reconstruction procedures (Section 5.5); the per-commit ledger appears in Supplemental Note C.

1.2 Related Work

Resource estimation and circuit optimization for the ECDLP. Quantum ECDLP resource estimates progressed from early circuit constructions [31, 52] to explicit

Toffoli estimates and windowed arithmetic [27, 54], followed by architecture-specific analyses and improved point-addition designs [3, 9, 22, 25, 36, 40, 59]. Google attests its unpublished circuit through a zero-knowledge proof [3], whereas several recent designs are open [22, 40, 59]. These constructions were each developed by one researcher or team. In contrast, our public competition reaches comparable operating points through contributions from independent human–agent teams and releases the resulting source history. TrailMix likewise provides inspectable circuits but follows one maintained implementation rather than a community search [64]. Our designs recombine reversible primitives and arithmetic architectures from this literature [5, 13, 32, 33, 35, 50].

Machine learning for circuit and algorithm optimization. AlphaTensor, AlphaDev, AlphaTensor-Quantum, and reinforcement-learning methods apply trained optimizers to algorithm or circuit search [19, 20, 41, 56], extending classical superoptimization [42, 58]. We instead use off-the-shelf agents, human steering, and a public leaderboard, without task-specific training. Because learned results may be rediscovered by conventional search or improved manually [30, 46], we explicitly audit provenance and distinguish invention from rediscovery.

Agents and autoresearch under a verifiable objective. FunSearch, AlphaEvolve, and The AI Scientist use generate–evaluate–iterate loops [38, 48, 55]; SWE-agent adds tool-driven software engineering [67], while verifiable-reward training uses exact checkers [15]. These systems run centrally. Open autoresearch instead lets independent human–agent teams share a public evaluator and repository. Because successful and failed submissions remain visible, we analyze the research process alongside its outputs.

Open, collaborative, and crowdsourced problem solving. Distributed problem solving appears in open-innovation contests, crowd science, collaborative mathematics, and games such as Foldit [12, 21, 26, 63]. Open autoresearch extends this model by placing AI agents alongside human participants, targeting frontier computational-science problems, and preserving the public search record rather than only the winning result.

1.3 Organization of the Paper

The remainder of the paper is organized as follows. Section 2 reviews the necessary background. Section 3 defines the challenge, evaluator, resource model, and correctness scope. Section 4 develops the open autoresearch paradigm, Section 5 presents the optimized circuits, principal techniques, resource trajectories, and reproducibility procedures, and Section 6 discusses limitations and responsible interpretation. Supplemental Note A, Supplemental Note B, and Supplemental Note C provide detailed trajectory analysis, an optimization catalog, and a source-commit census, respectively. Circuit-level analyses of the $Q \times T$-track submission `8e9c9a2`, the 825-qubit submission `b6f2b0a`, and the coherent

windowed-addition variant of `8e9c9a2` appear in Supplemental Note D, Supplemental Note E, and Supplemental Note F.

2 Preliminaries

2.1 Quantum Computing Background

Qubits and quantum registers. A *qubit* is the quantum analogue of a bit: a normalized vector in $\mathbb{C}^2$ spanned by the computational basis states $|0\rangle$ and $|1\rangle$. A *register* of m qubits lives in the 2^m-dimensional space spanned by the basis states $|x\rangle$ for $x \in \{0,1\}^m$, and its general (pure) state is a *superposition* $\sum_x \alpha_x |x\rangle$ with $\sum_x |\alpha_x|^2 = 1$. Measuring the register in the computational basis returns x with probability $|\alpha_x|^2$ and collapses the state onto $|x\rangle$. Throughout this paper registers encode 256-bit integers. These registers are most often the $\mathbb{F}_p$ coordinates of an elliptic-curve point. So "the accumulator holds a superposition of curve points" means that two 256-qubit coordinate registers are jointly in a superposition of basis states, each encoding one affine point. A quantum circuit can coherently transform a superposition of exponentially many computational-basis states, but their amplitudes cannot be read out individually: measurement yields only a sampled classical outcome. Quantum algorithms must therefore use interference to concentrate probability on outcomes that encode the desired global structure.

Reversible computation. During coherent portions of a quantum computation, the evolution of the quantum registers is governed by unitary operators [47]. Practical circuits may also include measurement and classically controlled feed-forward, which are described by quantum instruments rather than by a single unitary acting on the algorithmic registers. Because unitary transformations are invertible, coherent arithmetic must be expressed using reversible gates that preserve sufficient information to recover their inputs. The resulting constraints are not unique to quantum computation: analogous requirements arise in classical reversible computing and in constant-time cryptographic implementations designed to resist timing attacks. They nevertheless become especially consequential when data may be in superposition. First, control flow depending on quantum data must be represented by coherently controlled operations within a fixed circuit schedule. An inherently data-dependent procedure, such as the variable-length GCD iteration used for modular inversion, must therefore be unrolled or padded to a fixed bound or iteration budget, with each branch replaced by controlled operations, yielding a circuit whose shape covers all supported inputs [54]. Second, intermediate quantum results cannot be irreversibly discarded while they may remain entangled with the live state; they must instead be reversibly uncomputed or removed through measurement-based uncomputation with appropriate outcome-dependent corrections.

Uncomputation and cleanliness. Scratch (*ancilla*) qubits hold intermediate values during a computation. If an ancilla is discarded or reused while still correlated

with the data, it stays *entangled* with the output registers and destroys the interference on which the surrounding algorithm depends. The standard remedy, due to Bennett [4], is to *uncompute*: after the result is copied out, the computation of the scratch is run in reverse so every ancilla returns to $|0\rangle$, roughly doubling the gate cost. *Measurement-based uncomputation* [23] replaces part of this inverse computation by measuring ancillas in the X basis and applying classically conditioned Clifford corrections; it saves Toffoli gates, but an incomplete correction leaves a residual *relative phase* on surviving branches. Beyond computing the right classical function, a circuit intended to run inside a larger coherent algorithm must therefore also be clean in two quantum senses: every ancilla returns to $|0\rangle$, and no residual phase remains. The benchmark evaluator of Section 3 checks these as three separate channels; the two quantum channels need dedicated checks precisely because they are invisible to purely classical testing of input–output behavior.

Quantum table lookup. A recurring primitive is reading a *classical* table at a *quantum* address: given precomputed constants $\mathcal{T}[0], \ldots, \mathcal{T}[2^w - 1]$, a lookup (often called QROM, for quantum read-only memory) implements $|j\rangle|0\rangle \mapsto |j\rangle|\mathcal{T}[j]\rangle$. A 2^w-entry lookup costs $O(2^w)$ Toffoli gates, while the subsequent *unlookup* can be made substantially cheaper by measurement-based techniques [24, 36]. The canonical usage pattern is *lookup* $\rightarrow$ *use* $\rightarrow$ *unlookup*: the table value is loaded into a temporary register, consumed by the arithmetic that needs it, and immediately uncomputed, so the loaded data is live only briefly and can be scheduled away from the circuit's qubit high-water mark. This pattern is the interface through which windowed variants of Shor's algorithm (Section 2.2) consume elliptic-curve point tables, and it reappears in this paper wherever a circuit must handle data selected by a quantum index.

Fault-tolerant cost accounting. Large-scale cryptanalytic computations are expected to require quantum error correction, whose cost model is sharply nonuniform. In many surface-code architectures, Clifford operations are comparatively inexpensive, whereas non-Clifford gates consume costly distilled resource states [8]. For example, a conventional exact Clifford+T decomposition of a Toffoli gate uses seven single-qubit T gates, although measurement-assisted constructions can reduce this requirement [29]. Non-Clifford operations therefore capture an essential component of a circuit's nonlinear computational work. Although magic-state distillation has historically been expected to dominate physical cost, practical performance may also be limited by routing, storage access, decoding, classical feed-forward, and available parallelism. The executed Toffoli count is consequently a useful but architecture-dependent work proxy rather than a literal time axis. Similarly, quantum error correction gives each live logical qubit a substantial physical footprint, so the peak logical width Q lower-bounds the required logical storage without by itself determining the physical machine size. Writing T for the average executed Toffoli count, the challenge defines

$$S = Q \times T. \tag{1}$$

This product provides a compact, spacetime-inspired benchmark that rewards reductions in both logical width and nonlinear work. It is not a physical-resource estimate: it omits circuit depth, parallelism, routing, memory access, error-correction overhead, and other architecture-specific costs. Finer-grained alternatives include Toffoli depth, active-volume models, and architecture-specific gate schedules. Nevertheless, S captures a central width–work trade-off while remaining inexpensive and straightforward to evaluate under fixed benchmark rules, which motivates its use here. Section 3 specifies the accounting convention, including the distinction between emitted and *executed* Toffoli gates.

Kickmix circuits. Generic quantum circuits can generate dense entanglement, making them hard to simulate on a classical computer. However, the circuits in this paper live in a restricted class, called *kickmix* [3], that can be checked efficiently classically. By definition, a kickmix circuit is built from only three kinds of operation [3]: reversible permutation gates (X, CNOT, and Toffoli), which map one computational-basis state to another; diagonal phase gates (such as Z, CZ, and CCZ), which leave a basis state fixed and only attach a phase to it; and mid-circuit measurement in the X basis followed by classically conditioned phase corrections, which is the measurement-based uncomputation described above. What these operations have in common is that none of them spreads a basis state across the full 2^m-dimensional space. A computational-basis input therefore stays on a sparse support of only a few basis states throughout the circuit, so the circuit never builds the dense entanglement that makes general quantum computation hard to simulate. This is what lets the evaluator of Section 3 verify value correctness, ancilla cleanliness, and phase cleanliness on every test input rather than sampling or trusting the submitter.

2.2 Elliptic Curve Cryptography and Quantum Attacks

Elliptic curves and the group law. An elliptic curve [62] over a prime field $\mathbb{F}_p$ (in short Weierstrass form, $p > 3$) is the set of points (x, y) satisfying $E : y^2 = x^3 + ax + b$, $4a^3 + 27b^2 \neq 0$, together with a point at infinity $\mathcal{O}$. The points form an abelian group under the chord-and-tangent law with identity $\mathcal{O}$: for a point $R = (x_R, y_R)$ and another point $A = (x_A, y_A)$ with $A \neq \pm R$, their sum $R' = (x_{R'}, y_{R'}) = R + A$ is

$$\lambda = \frac{y_R - y_A}{x_R - x_A}, \qquad x_{R'} = \lambda^2 - x_R - x_A, \qquad y_{R'} = \lambda(x_A - x_{R'}) - y_A, \tag{2}$$

with $\lambda = (3x_R^2 + a)/(2y_R)$ for doubling ($A = R$). All arithmetic is in $\mathbb{F}_p$. The division in λ is a *modular inverse* $u \mapsto u^{-1} \bmod p$. Classically, it is a minor cost, but in the reversible setting of Section 2.1 it becomes the dominant expense of a point addition, because inversion has no branch-free single-pass circuit and must be realized as an unrolled, input-oblivious GCD iteration that is then uncomputed (Section 3). This paper concerns `secp256k1`, the curve $y^2 = x^3 + 7$ over $p = 2^{256} - 2^{32} - 977$ [10], whose group of $\mathbb{F}_p$-rational points is cyclic of

256-bit prime order r, generated by a standardized base point G. Throughout, $n = 256$ denotes the field bit-width; for `secp256k1` the group order satisfies $\lceil \log_2 r \rceil = n$ as well, so the exponent registers are likewise n qubits wide.

ECDLP and ECDSA. The *elliptic-curve discrete logarithm problem* (ECDLP) asks, given the generator G and a public point $P = [k]G$, to recover the scalar k. The forward direction costs $O(n)$ point additions, but the best known classical inverters (Pollard's rho, baby-step giant-step) require $\Theta(\sqrt{r}) \approx 2^{128}$ group operations for `secp256k1`, which is far beyond feasibility. This asymmetry underpins ECDSA [28], whose security rests on the hardness of ECDLP: against a properly implemented scheme, the best known classical key-recovery attack is to solve ECDLP for the signer's private key. ECDSA over `secp256k1` is the signature scheme of Bitcoin, Ethereum, and much of the surrounding blockchain ecosystem, which is why concrete quantum resource estimates for this specific curve carry direct economic significance.

Shor's algorithm for ECDLP. Shor's algorithm extends to discrete logarithms in finite abelian groups, including elliptic-curve groups, and therefore solves ECDLP in polynomial time [61]. Proos and Zalka subsequently provided the first detailed quantum circuits and concrete resource analysis for ECDLP over prime-field elliptic curves [52]. In the idealized hidden-subgroup formulation, the exponent registers range over $\mathbb{Z}_r$, where r is the order of G, and the probe function is

$$f(u, v) = [u]G + [v]P = [u + kv]G. \tag{3}$$

Because $f(u - k, v + 1) = f(u, v)$, the private key is encoded by the period direction $(-k, 1)$. Preparing a uniform superposition over $\mathbb{Z}_r^2$, evaluating

$$U_f : |u, v\rangle\,|\mathcal{O}\rangle \longmapsto |u, v\rangle\,|[u]G + [v]P\rangle\,, \tag{4}$$

and applying inverse Fourier transforms over $\mathbb{Z}_r$ produces characters in the annihilator of this period. Thus, in the idealized model, the measured pair (c, d) satisfies

$$d \equiv kc \pmod{r}, \tag{5}$$

from which $k = d\,c^{-1} \bmod r$ can be recovered when c is invertible.

A practical implementation stores the exponent values in n-qubit registers whose computational basis ranges over $\mathbb{Z}_{2^n}$. Applying $H^{\otimes n}$ prepares each register in a uniform superposition. Because 2^n generally differs from the subgroup order r, the measured Fourier outcomes do not satisfy the ideal relation $d \equiv kc \pmod{r}$ exactly; instead, they cluster near values consistent with that relation. Classical postprocessing extracts candidate values of k, which are verified by checking whether $P = [k]G$. A semiclassical Fourier transform can reduce the required control-register storage, but it does not literally collapse two existing registers into one qubit. Rather, it evaluates the Fourier transform iteratively by preparing a reusable control qubit, using it to control the elliptic-curve arithmetic, applying the appropriate phase rotation, measuring it, and then repeating

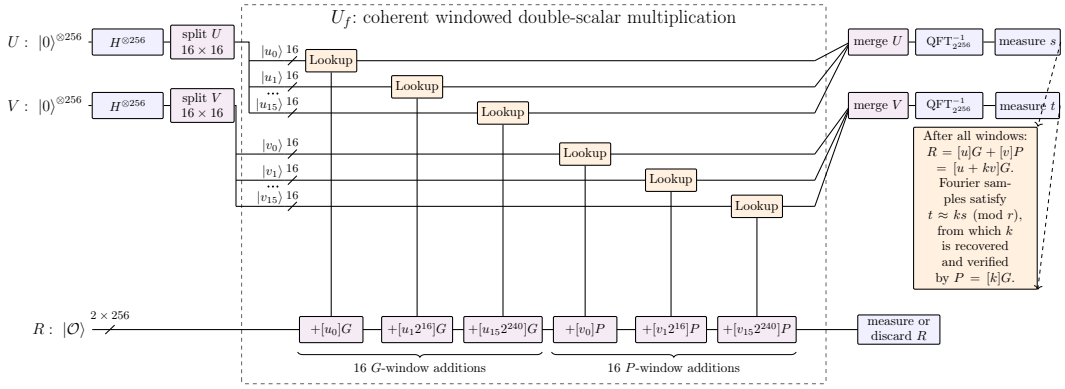


Fig. 3: Placement of the point-addition kernel within a power-of-two implementation of windowed Shor ECDLP. The coherent exponent registers are prepared over $\mathbb{Z}_{2^{256}}$ and split into 16-bit windows. Each window coherently selects a precomputed multiple of G or P and adds it to the accumulator, yielding $R = [u]G + [v]P$ before inverse power-of-two Fourier transforms and measurement. Unlike the idealized formulation over $\mathbb{Z}_r^2$, the measured Fourier samples need not satisfy the annihilator relation exactly; they concentrate near it and require classical postprocessing followed by verification of the recovered candidate k. The figure shows the coherent-register formulation; an iterative semiclassical implementation can instead serialize the Fourier controls using a recycled control qubit. The braces show the 32 conceptual window additions, while boundary optimizations reduce the estimate used in this paper to 28 point-addition calls. Submission `8e9c9a2` implements the mixed point-addition kernel, and Section 5.2.3 implements and evaluates a windowed-addition-compatible variant that coherently supplies the addend through QROM. Instantiating the complete 28-call window schedule, all shifted tables, the Fourier transforms, and the classical postprocessing remains future work.

the process. In either formulation, the controlled elliptic-curve arithmetic dominates the resource cost: evaluating U_f requires scalar multiplication by both G and P and therefore accounts for most of the logical width and Toffoli count. Figure 3 shows the coherent-register formulation, while the windowed point additions used within its scalar multiplications are detailed in Figure 4.

Two methods for scalar multiplication. The two scalar multiplications inside U_f, namely $[u]G$ and $[v]P$, decompose into sequences of point additions that follow one of two standard methods: *double-and-add* or *windowing* (Figure 4). These methods expose different point-addition interfaces, a distinction that matters throughout this paper.

In the original double-and-add method [52,54], let (s, B) denote either (u, G) or (v, P) and write $s = \sum_i s_i 2^i$. At step i, the fixed, classically known point

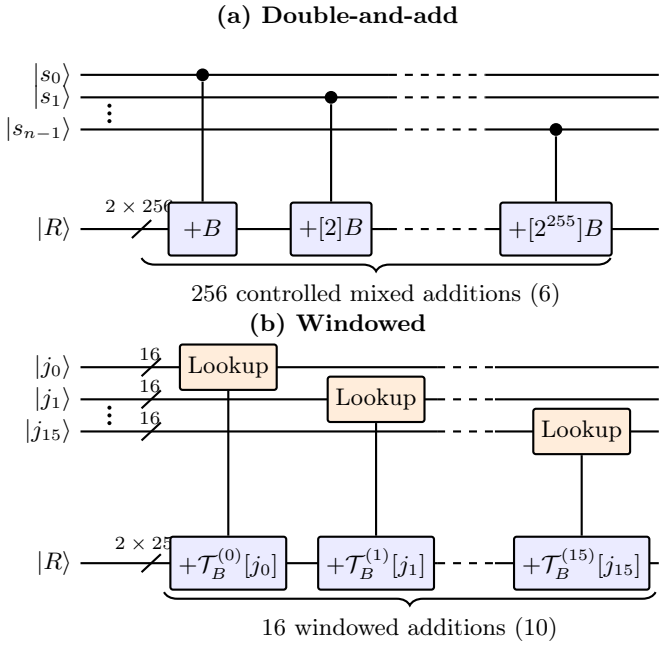


Fig. 4: The two point-addition interfaces used to compute one scalar multiplication $[s]B$, where (s, B) denotes either (u, G) or (v, P). **(a)** In the double-and-add approach, each scalar bit s_i controls a mixed addition of the fixed, classically known point $[2^i]B$ to the accumulator $|R\rangle$; this addend is fixed at circuit-construction time and does not occupy a quantum register. **(b)** In the windowed approach, the scalar is divided into 16-bit words $j_i = s_{16i:16i+16}$. Each $|j_i\rangle$ indexes a QROM lookup into the precomputed table $\mathcal{T}_B^{(i)}$, and the selected point $\mathcal{T}_B^{(i)}[j_i]$ is added coherently to $|R\rangle$. The selected addend is therefore quantum data entangled with the window register.

$A_i = [2^i]B$ is added to the accumulator R, controlled by the scalar bit s_i. The required primitive is therefore a controlled mixed elliptic-curve point addition

$$U_{A_i} : |s_i\rangle\,|R\rangle \longmapsto |s_i\rangle\,|R + [s_i]A_i\rangle\,, \tag{6}$$

whose addend A_i is a classical constant fixed at circuit-construction time. Across the two n-qubit exponent registers U and V of the probe function (3), this primitive is executed $2n$ times [54]. Writing $\mathrm{PA}_n^{\mathrm{Toff}}$ for the Toffoli count of one point addition, the full double-and-add Shor ECDLP estimate is

$$\mathrm{ECDLP}_n^{\mathrm{Toff}} \approx 2n \cdot \mathrm{PA}_n^{\mathrm{Toff}}. \tag{7}$$

The windowed approach [27] trades point additions for table lookups. Rather than consuming one exponent bit at a time, it groups the bits into w-bit values called *windows*. Following previous work [3, 36], we use $w = 16$. For either

$(s, B) = (u, G)$ or $(s, B) = (v, P)$, with $w \mid n$, scalar multiplication can be written as

$$[s]B = \sum_{\substack{0 \le i < n \\ i \equiv 0 \pmod{w}}} [s_{i:i+w} 2^i]B, \tag{8}$$

where $s_{i:i+w}$ is the w-bit window of s beginning at bit i. For each window position i, let $B_i = [2^i]B$ and classically precompute the table

$$\mathcal{T}_B^{(i)} = \big[\, [0]B_i,\, [1]B_i,\, \ldots,\, [2^w - 1]B_i \,\big]. \tag{9}$$

For the quantum window value $j = s_{i:i+w}$, the lookup selects the addend $A_i = \mathcal{T}_B^{(i)}[j]$, and the windowed point-addition primitive realizes

$$U_{\mathcal{T}_B^{(i)}} : |j\rangle\,|R\rangle \longmapsto |j\rangle\,|R + A_i\rangle\,, \qquad A_i = \mathcal{T}_B^{(i)}[j]. \tag{10}$$

Here, the window index j is quantum data stored in exponent register U or V. Implementing this operation as 2^w separately controlled mixed adders, one for each table entry, would inflate the Toffoli cost by a factor of 2^w and is therefore prohibitive. Instead, the selected addend A_i is materialized using a QROM lookup at address $|j\rangle$, added coherently to the accumulator, and then uncomputed through the lookup/use/unlookup pattern of Section 2.1.

Grouping the exponent bits into windows reduces the conceptual number of point additions for the two scalar multiplications from $2n$ to $2n/w$. The resource estimate applies two additional schedule-boundary optimizations. First, the point selected by the initial lookup is written directly into the accumulator, avoiding one point addition. Second, classical postprocessing permits three terminal point additions to be omitted. The resulting windowed schedule therefore contains $2n/w-4$ point-addition calls. These four savings are not consequences of windowing itself: analogous optimizations also shorten the non-windowed schedule. For $n = 256$, classical postprocessing saves roughly 48 bitwise additions, while direct lookup of the starting point saves roughly another 16. Including an additional QROM cost of $3 \cdot 2^w$ Toffoli gates per retained windowed addition gives the full Shor ECDLP estimate [3, 36]

$$\mathrm{ECDLP}_n^{\mathrm{Toff}} \approx \left(\mathrm{PA}_n^{\mathrm{Toff}} + 3 \cdot 2^w\right)\left(\frac{2n}{w} - 4\right). \tag{11}$$

For $n = 256$ and $w = 16$, windowing first reduces the conceptual schedule from 512 bitwise additions to 32 windowed additions; direct initialization and classical postprocessing then reduce the implemented estimate to 28 point-addition calls, each using tables with 2^{16} entries. Figure 4 contrasts the two point-addition interfaces at the circuit level.

Elliptic-curve point-addition blueprint. Elliptic-curve point addition is defined by the group law in (2). Previous constructions [52, 54, 59] realize this operation through the sequence of in-place field-arithmetic transformations given in Algorithm 1.

Algorithm 1 In-place generic addition of a classically specified addend A to a quantum accumulator R, realizing $R' = R + A$. Registers X and Y hold field elements and are recycled in place.

Input: $R = (x_R, y_R) \in E(\mathbb{F}_p)$ and $A = (x_A, y_A) \in E(\mathbb{F}_p)$.
Assumptions: $R, A \neq \mathcal{O}$ and $R \notin \{A, -A, -2A\}$; equivalently, $x_R \neq x_A$ and $x_A \neq x_{R'}$, where $R' = R + A$.
Output: The coordinate registers contain $R' = (x_{R'}, y_{R'}) = R + A$.

1:	$X \leftarrow x_R - x_A; \quad Y \leftarrow y_R - y_A$	$\triangleright\ X = d_x,\ Y = d_y$
2:	$Y \leftarrow YX^{-1}$	$\triangleright\ Y = \lambda$
3:	$X \leftarrow X + 3x_A$	$\triangleright\ X = x_R + 2x_A$
4:	$X \leftarrow X - Y^2$	$\triangleright\ X = x_A - x_{R'}$
5:	$Y \leftarrow YX$	$\triangleright\ Y = \lambda(x_A - x_{R'})$
6:	$Y \leftarrow Y - y_A$	$\triangleright\ Y = y_{R'}$
7:	$X \leftarrow x_A - X$	$\triangleright\ X = x_{R'}$

As observed by Proos and Zalka [52, Sec. 4.3], the key to an in-place reversible implementation is that the slope λ can be recovered from either the input points R and A or the output point $R' = R + A$:

$$\lambda = \frac{y_R - y_A}{x_R - x_A} = -\frac{y_{R'} + y_A}{x_{R'} - x_A} \pmod{p}. \tag{12}$$

The coordinate updates in Algorithm 1 follow directly from the group law and (12). Because $x_{R'} = \lambda^2 - x_R - x_A$ and X initially holds $x_R - x_A$, Steps 3–4 give $X = (x_R - x_A) + 3x_A - \lambda^2 = x_A - x_{R'}$. The final update in Step 7, $X \leftarrow x_A - X$, therefore produces $x_{R'}$. Similarly, (12) implies $\lambda(x_A - x_{R'}) = y_{R'} + y_A$, so Steps 5–6 transform Y from λ into $y_{R'}$. Further details on the quantum operations used to implement these transformations are provided in Supplemental Note B: List of Agent-Identified Optimizations.

Dialog-GCD record and replay. An efficient way to implement the in-place multiplication and division in Algorithm 1 is Schrottenloher's record-and-replay Euclidean construction [59], based on the dialog representation of Khattar et al. [33]. Rather than maintaining Bézout coefficients throughout the Euclidean descent, its record phase evolves only two shrinking integers and stores the branch decisions needed for later reconstruction. In this paragraph and the related optimization subsections, u and v locally denote Euclidean operands rather than the exponent variables of Shor's algorithm.

Algorithm 2 provides the pseudo code. When $b_i = 0$, v is already even; when $b_i = 1$, the optional swap leaves two odd operands and their difference is even. Thus, the final division by two is exact in either case. Because x may be quantum data, the symbols σ_i form a coherent quantum transcript τ and cannot be measured or discarded.

For the transcript generated by a nonzero x, let D_x denote the resulting linear Euclidean transformation. Successful convergence gives $D_x(p, x) = (1, 0)$. Reverse traversal of τ applies the corresponding modular updates to the Bézout

Algorithm 2 Dialog-GCD record phase, based on Schrottenloher [59, Alg. 2].

Input: $(u, v) = (p, x)$ with $x \in \mathbb{F}_p^*$ and odd u; fixed ordinary-divstep budget L_{div}
Output: coherent transcript $\tau = (\sigma_0, \ldots, \sigma_{L_{\text{div}}-1})$

$\tau \leftarrow ()$
for $i = 0, \ldots, L_{\text{div}} - 1$ **do**
  $b_i \leftarrow [v \text{ is odd}]$ ▷ subtraction flag
  $s_i \leftarrow b_i \wedge [u > v]$ ▷ swap flag
  $\sigma_i \leftarrow (b_i, s_i)$; $\tau \leftarrow \tau \parallel \sigma_i$
  if s_i **then**
    $(u, v) \leftarrow (v, u)$
  end if
  if b_i **then**
    $v \leftarrow v - u$
  end if
  $v \leftarrow v/2$
end for
Successful endpoint: $(u, v) = (1, 0)$

or payload registers, yielding $D_x(0, y) = (yx^{-1}, 0)$ and $D_x^{-1}(y, 0) = (0, yx)$. Stage 2 uses $x = d_x$ and $y = d_y$ to obtain $\lambda = d_y d_x^{-1}$, while Stage 5 uses the inverse transformation for in-place multiplication. The optimized iteration schedule and transcript representation used by the final circuit are described in Sections 5.3.1 and 5.3.2.

3 The ECDSA.Fail Challenge and the Benchmark

ECDSA.Fail [17, 18] is an open competition to minimize the resources of a reversible `secp256k1` point-addition circuit; the curve underlies Bitcoin, Ethereum, and related blockchain systems [43, 66]. Submissions are scored by the fault-tolerant spacetime proxy in (1) and accepted only if they are value-correct, ancilla-clean, and phase-clean on an evaluator-generated test set. This section defines the task and its relationship to Shor's algorithm, the circuit model, cost metric, and evaluator. Throughout, $n = 256$ is the field bit-width and $p = 2^{256} - 2^{32} - 977$ is the `secp256k1` prime.

3.1 The Point-Addition Task

As described in Section 2.2, the cost of applying Shor's algorithm to `secp256k1` is dominated by the two scalar multiplications, $[u]G$ and $[v]P$, in the probe function (3). These decompose into repeated elliptic-curve point additions, making point addition a principal contributor to the logical width and Toffoli count and motivating ECDSA.Fail to isolate and optimize this primitive.

For `secp256k1`, $E : y^2 = x^3 + 7$ over $\mathbb{F}_p$, the benchmark supplies a quantum accumulator $R = (x_R, y_R)$ in two 256-qubit registers and a classically specified

addend $A = (x_A, y_A)$. It asks the circuit to implement the reversible in-place map

$$U_A : |x_R\rangle\,|y_R\rangle \longmapsto |x_{R'}\rangle\,|y_{R'}\rangle\,, \qquad R' = R + A, \tag{13}$$

using the generic affine group law in (2). Computing its slope requires $(x_R - x_A)^{-1} \bmod p$, the dominant reversible operation. The circuit's generic-affine domain is $R, A \neq \mathcal{O}$ and $R \notin \{A, -A, -2A\}$. The cases $R = A$ and $R = -A$ make the input-side denominator vanish, requiring point doubling or producing $\mathcal{O}$, respectively. The additional case $R = -2A$ gives $R' = -A$ and hence $x_A - x_{R'} = 0$, which lies outside the domain of the reversible in-place multiplication used later in the circuit. The evaluator explicitly filters identity inputs and $x_R = x_A$, but does not separately filter $R = -2A$; the latter occurs with probability approximately $1/r$ under random-scalar sampling and therefore has negligible probability in the finite validation set. Circuit correctness is nevertheless not claimed for this case. The quantum inputs `target_x` and `target_y` initially hold R, while the classical inputs `offset_x` and `offset_y` specify A; the circuit overwrites the target registers with R'. The scored primitive is the uncontrolled map in (13).

Compatibility with the full Shor circuit. Because the benchmark addend A is classically specified, (13) closely matches the mixed addition used in double-and-add (Section 2.2), where each call adds a fixed point $A_i = [2^i]B$ for $B \in \{G, P\}$. Double-and-add additionally controls each call on a scalar bit, so integration requires restoring the global control omitted by the benchmark. Standard techniques for doing so are given in [27, 54].

The windowed method presents a different interface. As discussed in Section 2.2, it instead selects each addend coherently through QROM, for example $A_i = \mathcal{T}_G^{(i)}[u_i]$ or $\mathcal{T}_P^{(i)}[v_i]$, making its coordinates live quantum data. The benchmark circuit is therefore not directly drop-in compatible and must adopt the QROM lookup/use/unlookup interface of Section 2.1. We implement this adaptation in Section 5.2.3: the resulting variant supplies the addend coherently from a quantum-addressed classical table and clears the lookup workspace by exact reversible replay, adding 11 logical qubits and approximately 29.6% average executed Toffoli per addition.

3.2 Circuit Model and Cost Metric

A submission represents a reversible point-addition map in the kickmix model of Section 2.1. Its serialized instruction set includes X, Z, CNOT, CZ, SWAP, CCX, CCZ, qubit reset, and HMR, a demolition measurement in the X basis. Standalone Hadamard gates and persistent $|+\rangle$-state preparation are excluded. This restriction is essential to the tractability of verification: CCX together with the Hadamard gate is universal for quantum computation [1], so allowing unrestricted Hadamards would make the language expressive enough to encode general BQP computations, beyond the scope of the benchmark's efficient classical verifier. Indeed, even reusable $|+\rangle$-state preparation, combined with the

Type	What is checked	Why it is required
`cls`	Outputs equal the affine sum $R' = R + A$ on every test.	Verifies the intended classical map.
`anc`	All scratch and non-output registers return to $\|0\rangle$.	Prevents garbage entanglement with the outer computation.
`pha`	Surviving branches retain no relative phase after uncomputation or measurement-based cleanup.	Preserves the interference required by Shor's algorithm.

Table 1: Correctness checks; a clean submission reports 0/0/0 on every test.

existing kickmix operations and HMR, can synthesize a standalone Hadamard. Hadamard is therefore permitted only within HMR, where it is immediately followed by destructive measurement and does not leave a reusable coherent superposition. Each executed CCX or CCZ counts as one Toffoli-equivalent operation. Under the fault-tolerant cost model of Section 2.1, Toffoli count serves as an architecture-dependent proxy for non-Clifford work, while peak qubit width measures logical space, so the benchmark reports two resources:

- **Peak qubits** Q: the maximum number of logical qubits simultaneously live at any point in the circuit.
- **Average executed Toffoli** T: the expected number of executed Toffoli-equivalent gates, averaging data-dependent firing probabilities over the evaluator's input distribution [3]. It counts Toffoli operations, not magic-state T gates, and differs from the static gate count and circuit depth.

The benchmark scores the product of these two quantities,

$$S = Q \times T, \tag{14}$$

Lower scores are better. The product approximates fault-tolerant spacetime volume, with Q representing space and T the non-Clifford time or distillation load. Because point addition recurs throughout Shor's algorithm, improvements to this primitive reduce the whole-algorithm resource estimate. In addition to this headline metric, the challenge also maintains a low-qubit track that minimizes Q independently of T for qubit-constrained settings. Together, the two objectives chart the (Q, T) Pareto frontier, whose circuits cannot improve one resource without increasing the other; much of this work seeks to advance that frontier.

3.3 Correctness, Verification, and Scoring

Acceptance requires passing the evaluator's three correctness checks (Table 1): `cls` verifies the classical function on sampled inputs, while `anc` and `pha` detect unclean ancillas and residual phases that classical tests cannot observe but that would corrupt Shor's interference.

Test points. Following the verifiable fuzz-testing methodology of [3], the evaluator avoids a fixed public test set by seeding SHAKE256 with the serialized submitted circuit and expanding its output into 9,024 curve points. The resulting test set is deterministic but circuit-dependent and becomes known only after the circuit is fixed. Its support-selection and nonce-hunting limitations are discussed in Section 6.

Approximation and nonce hunting. Exactness is not required: the reference circuits of Babbush et al. [3] and Schrottenloher [59] are also approximate, with a bounded failure fraction of at most approximately $2^{-13.3}$ [59]. Submissions can therefore trade accuracy for lower $Q \times T$ by truncating comparisons, narrowing carries, or removing rarely executed gates.

Because the evaluator derives its 9,024 test inputs from a SHAKE256 hash of the serialized operation stream, functionally equivalent emitted circuits can induce different validation sets. The implementation encodes a 48-bit nonce in a fixed 96-operation identity tail: for each bit b_i, it applies Pauli X twice to either `target_x`[0] or `target_x`[1], according to the value of b_i. Since $X^2 = I$, this preserves the circuit function, Toffoli count, and peak width, while the changed target operands alter the hash. Submitters can search these nonces for an error-free test set, or clean validation *island*. For a circuit with error rate ϵ, the clean-nonce density, or *landability*, is approximately $(1-\epsilon)^{9024}$ under the random-oracle model and therefore decreases exponentially with ϵ. In particular, $(1-0.01)^{9024} \approx 2^{-130.8}$, so hiding a 1% error rate would require more than 2^{128} candidates on average and is infeasible within the 48-bit nonce space [3]. To assess improvements independently of nonce-selected support, we reevaluate a longitudinal sample on 50,000 common pseudorandom inputs and report $Q \times T/\hat{p}$, where $\hat{p}$ is the empirical success rate; because $\hat{p}$ remains near one, the retry-adjusted trajectory closely follows the raw score (Section 5.4.1).

Anchors. The baseline has $Q = 2{,}715$ and $T = 3{,}960{,}753$, giving $Q \times T \approx 1.07 \times 10^{10}$. External space-optimized reference points include Schrottenloher's $Q = 1{,}192$, $T = 2^{21.19} \approx 2.4M$ circuit [59] and the Babbush et al. estimate of $Q = 1{,}175$, $T = 2^{21.36} \approx 2.69M$ [3]. Section 5 analyzes the improvement trajectory, Pareto frontier, and cross-work comparisons.

3.4 Formulating the Circuit Optimization Problem

Under the circuit model and evaluator above, ECDSA.Fail solves

$$\min_{C \in \mathcal{K}} Q(C)T(C) \qquad \text{subject to} \qquad \mathrm{cls}(C) = \mathrm{anc}(C) = \mathrm{pha}(C) = 0,$$

where $\mathcal{K}$ contains the circuits admitted by the kickmix language and locked harness. The task combines valid reversible-circuit synthesis with minimization of nonlinear gate cost and live quantum memory. Related function, unitary, and state minimization variants admit QCMA formulations and search-to-decision reductions [11]; exact reversible Toffoli minimization is NP-hard [65],

while subspace-equivalent minimization of several gate counts and depths is co-NQP-hard [34]. Width optimization adds a hard scheduling problem: under spooky pebbling, minimizing quantum space on general dependency DAGs is PSPACE-hard to approximate, and the corresponding decision problem is PSPACE-complete [35, 53]. These results make efficient exact global optimization implausible and motivate Section 4, where agent-driven optimizations accumulate evaluator-verified improvements without exhaustive synthesis.

4 The Open Autoresearch Paradigm

This section defines open autoresearch and its discovery loop (Section 4.1), describes its ECDSA.Fail implementation (Section 4.2), summarizes observed participant practices (Section 4.3), and consolidates them into a reusable framework (Section 4.4).

4.1 Overview of Open Autoresearch

We define *Open Autoresearch* as a process in which a heterogeneous community of humans and AI agents addresses a hard problem formulated as a measurable objective [38,48,55]. An *AI agent* is an LLM coupled to tools and persistent state, enabling it to act, observe machine feedback, and revise its next step [37, 57, 60, 68]. *Autoresearch* delegates repeated hypothesis generation, implementation, and evaluation to these agents under human supervision. Its enabling condition is a *verifiable objective*: a computable measure that efficiently scores candidates and checks their validity. ECDSA.Fail instantiates this paradigm for quantum-circuit optimization, but the same structure applies to other domains [19, 41].

Participants may range from domain experts to enthusiasts with unrelated backgrounds, combining human judgment with agents that survey literature, test candidates, and record successes and failures. The model is open [63]: participants need not know one another, share an institution, or coordinate beforehand.

Figure 5 depicts the generic discovery loop. Participants select a base, propose a change, and submit it for scoring and validation. The base may be the current best, another Pareto point, or an off-frontier design chosen to escape a local optimum. Valid improvements become public bases for subsequent work; failed candidates are revised and their diagnoses retained as negative evidence. Shared objectives, evaluators, and artifacts thereby turn independent attempts into a cumulative search.

Optional communication through email, forums, messaging platforms, and GitHub accelerates the spread of new techniques. Independent branches using different models, tools, backgrounds, and objectives also make the search resilient to plateaus: an existing or newly joined contributor may unlock progress after the dominant route stalls, after which the accepted improvement becomes available for community-wide adoption and refinement.

Public, verified optimization leaderboards predate open autoresearch. For example, communities such as Opus Magnum already maintain cumulative, multi-

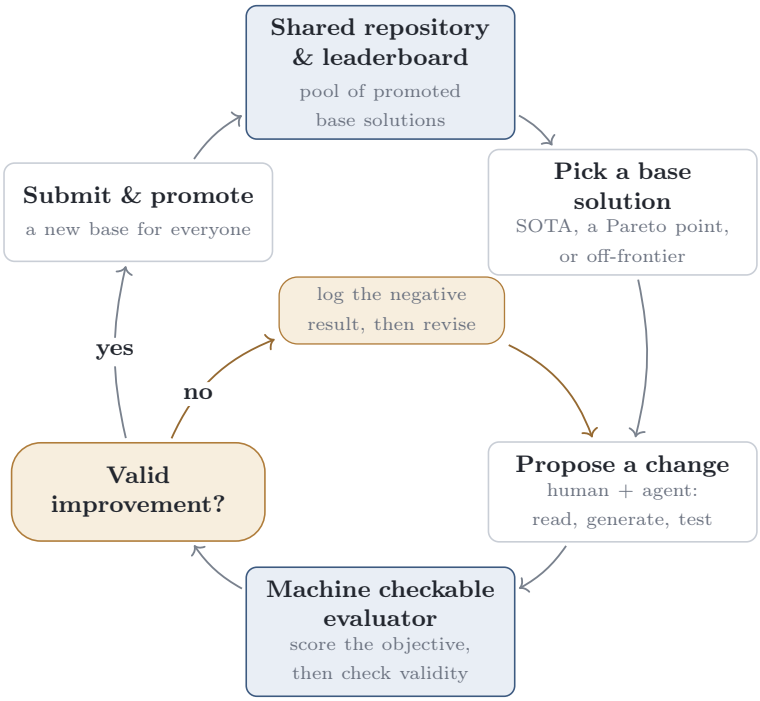


Fig. 5: The generic *open autoresearch discovery loop*. Independent human–agent teams select a current-best, Pareto, or deliberately off-frontier base from the shared repository, propose a change, and submit it to a machine-checkable evaluator. Verified improvements become shared bases; rejected candidates are logged with their diagnoses and revised. This makes the search cumulative without requiring prior coordination beyond the shared objective, evaluator, and public artifacts.

objective records of submitted solutions [6]. The distinction here is the integration of tool-using AI agents with a machine-checkable scientific evaluator, versioned source and provenance, and shared experiment memory that includes contextualized failures. Existing leaderboard communities could adopt agent-readable evaluator interfaces, automated change summaries, and structured negative-result records to support more autonomous cumulative search.

4.2 The ECDSA.Fail Open Autoresearch Harness

Launched by Eigen Labs in late May 2026, ECDSA.Fail applies open autoresearch to a reversible `secp256k1` mixed point-addition circuit. Its primary score is $S = Q \times T$ (Section 3.2), supplemented by low-qubit and low-Toffoli tracks that expose other Pareto operating points. The benchmark and results appear in Sections 3 and 5.

The harness implements the discovery loop through a shared repository, locked evaluator, command-line interface, and public leaderboard. Under fixed scoring rules, the command-line tool reports Q, T, and $Q \times T$ and checks classical output, ancilla cleanliness, and phase correctness. Promoted improvements add their score, source, participant attribution, and declared AI model to the public record, allowing others to synchronize or selectively adopt their techniques.

Participation requires only repository access; Slack and Telegram provide optional discussion and debugging. Contributors may edit circuits directly or delegate implementation and testing to coding agents. Over approximately eight weeks, more than one hundred participants using diverse models produced over four hundred promoted submissions while pursuing frontier, independent, and alternative-resource branches.

The harness thus provides a stable verification boundary and a cumulative record of accepted circuits and provenance. The trajectory combined incremental refinements with architectural changes to inversion, encoding, and register allocation. Section 5 analyzes these circuits, while Section 4.3 examines the practices that produced them.

4.3 ECDSA.Fail Autoresearch Techniques

This subsection examines how ECDSA.Fail participants organized AI agents to propose, implement, and verify circuit optimizations. It draws on contributed harnesses, prompts, scripts, experiment logs, and public discussions; representative raw harnesses and scaffolds are available in this supplemental repository.

4.3.1 Overview of the Autoresearch Scaffold and Methodology Building on the challenge harness described above, participants added scripts for parameter sweeps, small-domain exhaustive tests, counterexample-guided search, regression testing, and experiment tracking. Git worktrees or isolated clones kept concurrent experiments separate from the accepted baseline and made candidates reproducible.

Two agent-organization patterns recurred. In the *research-and-engineering* pattern, research agents generated hypotheses and ran inexpensive experiments, while an engineering agent converted promising results into clean patches. In the *orchestrator-and-workers* pattern, a coordinator assigned independent mechanisms to narrowly scoped agents and synthesized their findings. Both limited individual-agent context while preserving shared state through hypothesis queues, iteration logs, scoreboards, and regression records. Human instructions ranged from high-level objectives to detailed specifications of roles, branches, validation stages, and synthesis procedures.

A common workflow also emerged. Participants synchronized with the public frontier, inspected source changes and submission notes, and selected a baseline for the target resource regime. Agents proposed circuit-level changes, estimated or measured their local costs, tested them cheaply, and then evaluated the complete point-addition circuit. Validation progressed from smoke tests through re-

gression checks to the official evaluator, with failures classified by classical output, phase, and ancilla cleanliness. Promising candidates were rebuilt cleanly and compared with the live frontier before submission.

Experiment records included rejected variants, their assumptions, and observed failure modes, allowing later agents to avoid repetition or revisit ideas whose prerequisites had changed. Humans generally remained outside the implementation loop, choosing resource regimes, allocating compute, supplying domain knowledge, redirecting stalled searches, and judging whether score changes reflected genuine improvements. The principal practices for parallel search, failure diagnosis, and human–agent collaboration are discussed next.

4.3.2 Observed Autoresearch Practices Within the common scaffold described above, several practices recurred across successful research efforts. These methods span the organization of agent labor, the selection and diagnosis of experiments, and the accumulation of knowledge across sessions and participants. These practices are descriptive rather than prescriptive: no participant used all of them, and successful efforts combined them in different ways. The value of the open framework is precisely that it permits heterogeneous search strategies.

- **Reusable and evolving agent skills.** Participants packaged recurring procedures as versioned skills that agents could invoke across experiments. These included general methods for autonomous iteration, literature review, and self-critique, as well as challenge-specific procedures for reducing peak width, lowering Toffoli count, diagnosing reversible-circuit failures, searching for admissible evaluator supports, and reconstructing frontier changes. When a new technique or failure mode was identified, its lessons could be incorporated into the relevant skill, allowing methodology to improve together with the circuit frontier.
- **Persistent loops and goals.** Participants often assigned long-running objectives through persistence mechanisms such as Claude Code's `/loop` command and Codex's `/goal` command, allowing agents to conduct repeated propose–implement–evaluate cycles with limited supervision. Persistence was paired with checkpoints that assessed novelty, progress, and diminishing returns. A stalled campaign could then be redirected, supplied with additional context, moved to another operating point, or terminated rather than continuing indefinitely along an unproductive route.
- **Parallel, role-separated, and clean-context agents.** Participants commonly separated structural research, implementation, scoring, and independent verification across agents. Orchestrators dispatched clean-context workers, each scoped to one hypothesis, literature question, or failure-mode analysis and asked to return a compact conclusion rather than an extended narrative. Some teams also conducted route bake-offs in which several candidate architectures received comparable initial budgets and only the most promising were explored further, approximating a beam search over circuit variants. Agents from different model families (e.g. GPT, Claude, Grok, GLM, and

Kimi) sometimes reviewed one another's conclusions, and every claimed improvement was treated as a hypothesis until independently reproduced in an isolated worktree. This organization broadened exploration while limiting context contamination and shared blind spots.

- **Summarizing and selectively adopting other participants' work.** Dedicated watcher agents compared adjacent public submissions, reconstructed the responsible source changes, and distinguished transferable circuit techniques from configuration-specific effects. Useful findings were summarized in research notes and incorporated into reusable skills. Active agents did not necessarily rebase whenever the frontier changed: participants often preserved independent branches and adopted a public improvement only when it strengthened the circuit family under investigation. This balance enabled rapid cross-pollination without collapsing the search onto a single trajectory.
- **Human steering and non-monotonic exploration.** Human intervention was most valuable in selecting research directions, supplying domain constraints, recognizing stagnation, and deciding whether a score change represented a meaningful circuit improvement. Participants sometimes directed agents to restructure an entire component while tolerating temporary increases in Q, T, or validation failures, and only afterward repair and optimize the new design. A related strategy asked what prerequisites would have to hold for a proposed optimization to work and then searched for transformations that established those prerequisites. Such non-monotonic exploration exposed circuit families that strict hill climbing would reject prematurely.
- **Shared experiment memory and structured reflection.** Participants maintained append-only records containing each hypothesis, implementation, resource measurement, failure mode, and proposed next step. Failed directions were documented together with their assumptions rather than declared universally invalid, allowing later agents to revisit them after changes in the circuit architecture or resource regime. Periodic reflection passes summarized what had worked, identified recurring obstacles, and transferred insights between circuit components. These records reduced duplicated effort and preserved knowledge across context resets, model changes, and participant boundaries.
- **Literature-guided hypothesis generation.** Agents searched, indexed, and summarized relevant academic and technical literature, extracting candidate mechanisms and known limitations. Historically, Khattar et al. [33] introduced the underlying record-and-replay dialog-GCD architecture. However, the challenge record contains no independent agent-derived transfer of this architecture to ECDLP point addition; submissions implementing dialog-GCD appeared only after Schrottenloher published its detailed ECDLP adaptation [59], which enabled substantial reductions in $Q \times T$. Schrottenloher's paper appeared in early June 2026, only days after the challenge began on May 30, 2026, when participation remained limited and sophisticated literature-search and cross-domain retrieval scaffolds had not

yet been developed. This chronology does not isolate whether the missed transfer resulted from model capability, limited search effort, differences in terminology and problem formulation, or immature retrieval tooling. It nevertheless identifies cross-domain retrieval and abstraction as important capabilities to strengthen. Once the connection became available, agents rapidly implemented, tested, and refined the architecture against the shared evaluator, illustrating the complementary roles of human research and agent-scale experimentation.

- **Staged validation and mechanism-level diagnosis.** Candidate changes were tested first on inexpensive instances and targeted invariants before being subjected to the complete evaluator. When a circuit failed, participants separated classical-output, phase, and ancilla-cleanliness errors and used first-failure traces, error histograms, comparison outcomes, and carry-width measurements to identify the responsible mechanism. The smallest faulty component was then repaired before further parameter or evaluator-support searches were attempted. This approach prevented computational effort from being spent compensating for an unidentified structural error.
- **Exact transformations and circuit fusion.** Participants frequently prompted agents to seek algebraically exact transformations that reduced cost without introducing additional approximation error. Fusion was particularly productive: adjacent arithmetic blocks were combined so that intermediate values, controls, or cleanup operations could be eliminated, for example by replacing a sequence of modular additions and negations with one equivalent constant update. Because such transformations preserve correctness for all inputs under their stated preconditions, they can be composed and evaluated without relying on support-specific correctness behavior.
- **Alternative search-and-distill strategies.** At least one participant used the evaluator to generate low-cost correct subcircuits and trained a smaller model on the resulting examples. Increasing model size, adding reinforcement learning, and eliciting additional reasoning did not overcome the observed performance plateau. This negative result suggested that tool access and evaluator-guided iteration may be more important for this task than scaling a model trained to reproduce a fixed procedure.

4.3.3 Why It Works and What We Have Learned The ECDSA.Fail experience highlights four conditions that supported sustained human–agent progress and delimit where open autoresearch is most applicable.

- **A hard-to-solve but efficiently verifiable objective.** Discovering efficient reversible circuits is difficult, but their resource usage and sampled correctness can be evaluated automatically and reproducibly. Agents can therefore explore speculative changes while a common evaluator rejects contract violations and measures successful candidates. This paradigm is less suitable when evaluation is unreliable, uninformative, or too expensive for repeated experimentation.

- **Cumulative search through openness and diversity.** Public circuits, diffs, notes, and reusable skills preserved both accepted improvements and failed experiments. Participants could adopt promoted circuits while maintaining independent branches and operating points. This combination proved important when progress stalled: apparent plateaus were repeatedly broken by existing or newly joined contributors using different models, tools, expertise, or circuit families. The resulting trajectory was therefore a sequence of public handoffs among partially independent searches rather than the uninterrupted optimization path of one participant or agent.
- **Complementary human and agent capabilities.** Agents excelled at implementation, repeated experimentation, local diagnosis, and incremental refinement. Humans selected objectives, interpreted resource behavior, challenged assumptions, and supplied architectural insights. In particular, the dialog-GCD route used the record-and-replay Euclidean architecture introduced by Khattar et al. [33]. Agents accelerated its implementation and refinement and occasionally identified structural transformations that produced substantial resource reductions. The evidence therefore supports a division of labor combining human conceptual direction with agent-scale execution.
- **The objective shapes the research trajectory.** A scalar score rewards immediate progress but may suppress architectures that require temporary regressions. Separate low-qubit and low-Toffoli tracks mitigated this tendency by exposing different regions of the (Q, T) Pareto frontier. Future systems could also reward frontier expansion, reproducible negative results, or promising structural changes. Evaluators and incentives therefore influence which discoveries are pursued and preserved.

Open autoresearch is thus best suited to problems with reliable, inexpensive evaluation, public intermediate artifacts, and enough freedom for participants to pursue independent routes.

4.4 Consolidated ECDSA.Fail Autoresearch Framework

The practices in Section 4.3 emerged independently and were combined differently across participants; no participant used the complete set. Algorithm 3 synthesizes them into two concurrent processes. Process A monitors the public frontier and research literature, incorporates useful findings into shared skills and memory, and permits human reprioritization or course correction. Process B runs a token-bounded candidate-development cycle that synchronizes with the public record, selects a frontier or off-frontier base, profiles its bottlenecks, and explores hypotheses in parallel using fresh-context agents and isolated worktrees.

The framework separates speculative generation from evidential acceptance. Workers may pursue aggressive rewrites, approximate constructions, or temporarily inferior intermediate designs, but each candidate undergoes classical-output, phase, and ancilla-cleanliness checks. Invalid branches are repaired or discarded; approximate candidates additionally undergo landability triage and

Algorithm 3 Consolidated ECDSA.Fail open autoresearch framework

Require: track $\in \{S, Q, T, \text{Pareto}\}$; agent token budget
Require: skills $\mathcal{S}$; memory $\mathcal{M}$
Require: shared repository and leaderboard $\mathcal{L}$

Process A: Asynchronous monitoring and steering

A1: **while** the campaign remains active **do**
A2: monitor $\mathcal{L}$ and relevant research literature
A3: incorporate useful findings into $\mathcal{S}, \mathcal{M}$
A4: **human-in-the-loop:** steer the agents as needed (reprioritize / course-correct)
A5: update $\mathcal{M}$ with human steering
A6: **end while**

Process B: Candidate-development cycle

B1: **while** agent token budget remains **do**
B2: synchronize with $\mathcal{L}$; refresh $\mathcal{S}, \mathcal{M}$
B3: choose a frontier or off-frontier base solution B
B4: profile B; generate hypotheses $\mathcal{H}$ from $\mathcal{S}, \mathcal{M}$
B5: **for all** $h \in \mathcal{H}$ in parallel **do**
B6: launch a fresh-context subagent with a worktree
B7: assign h and the current $\mathcal{S}$ to the subagent
B8: implement and refine h until stalled
B9: test cls/pha/anc; repair or discard
B10: for approximate candidates, triage landability
B11: search nonces only when viable
B12: run the full evaluator; record the outcome
B13: **end for**
B14: select the best validated candidate $C^\star$
B15: rebuild and remeasure $C^\star$
B16: submit $C^\star$ if it improves the frontier
B17: on promotion, publish $C^\star$ as a shared base
B18: distill all outcomes into $\mathcal{S}$ and $\mathcal{M}$
B19: **end while**

Output: promoted circuits, evolved skills, and contextualized experiment records.

nonce search only when viable. Surviving candidates are evaluated under the full benchmark, after which the best is rebuilt, remeasured, and submitted only if it advances the selected frontier.

Progress compounds both within and across participants. Parallel workers broaden each local search, while promoted circuits become shared bases for the community. Skills and experiment memory retain useful mechanisms, measured outcomes, and contextualized failures, allowing later searches to avoid repetition or revisit ideas under changed prerequisites. The framework is therefore not a mandatory strategy but a synthesis of observed practices adaptable to other problems with machine-checkable objectives, inexpensive experiments, and shareable intermediate artifacts.

Submission	Q	T	Status	Interpretation
`b6f2b0a`	825	489,161,900	Admitted	Lowest observed width.
`f1d8707`	826	479,337,428	Admitted	Difference from q825: −9.82M Toffoli.
`39a9b5f`	828	448,102,916	Admitted	Difference from q826: −31.23M Toffoli.
`a3b0148`	948	54,781,961	Admitted	Observed exit from the extreme-width cluster.
`4352cfb`	980	28,847,321	Admitted	Lowest observed T below 1,000 qubits.
`1c0e0e9`	1,133	1,460,511	Admitted	Bridge to the product-efficient regime.
`8e9c9a2`	1,151	1,299,453	Promoted	Product-score incumbent and first 50% crossing.
`a536a48`	1,158	1,292,651	Promoted	Lowest-T promoted point at the cutoff.

Table 2: Archive-conditioned Pareto set at the data cutoff. Adjacent differences are descriptive secants between distinct circuit families, not causal marginal effects. "Admitted" denotes an archived public clean-run report included under the manuscript's curated selection, not official promotion.

5 Results and Findings

This section presents the challenge's principal technical and empirical results. We summarize the final operating points, Pareto frontier, external comparisons, and supporting evidence (Section 5.1); describe the resulting circuit architectures (Section 5.2) and key optimizations (Section 5.3); trace their evolution across submissions (Section 5.4); and provide the artifacts and procedures for reproduction (Section 5.5).

5.1 Overall Summary of Results

As noted in Section 3.2, ECDSA.Fail ranks circuits by $S = Q \times T$, where Q is peak logical width and T is the average executed Toffoli count over the submission-dependent validation support. Because participants also optimized these resources separately, the submissions define a (Q, T) Pareto frontier rather than a single optimum. We present the baseline and final operating points (Section 5.1.1) and compare them with prior point-addition constructions (Section 5.1.2).

5.1.1 The Baseline and the Pareto Frontier at the Data Cutoff The baseline circuit, `30c8ded`, used 2,715 logical qubits and 3,960,753 average executed Toffoli gates. The best-scoring circuit, `8e9c9a2`, reduced these values to

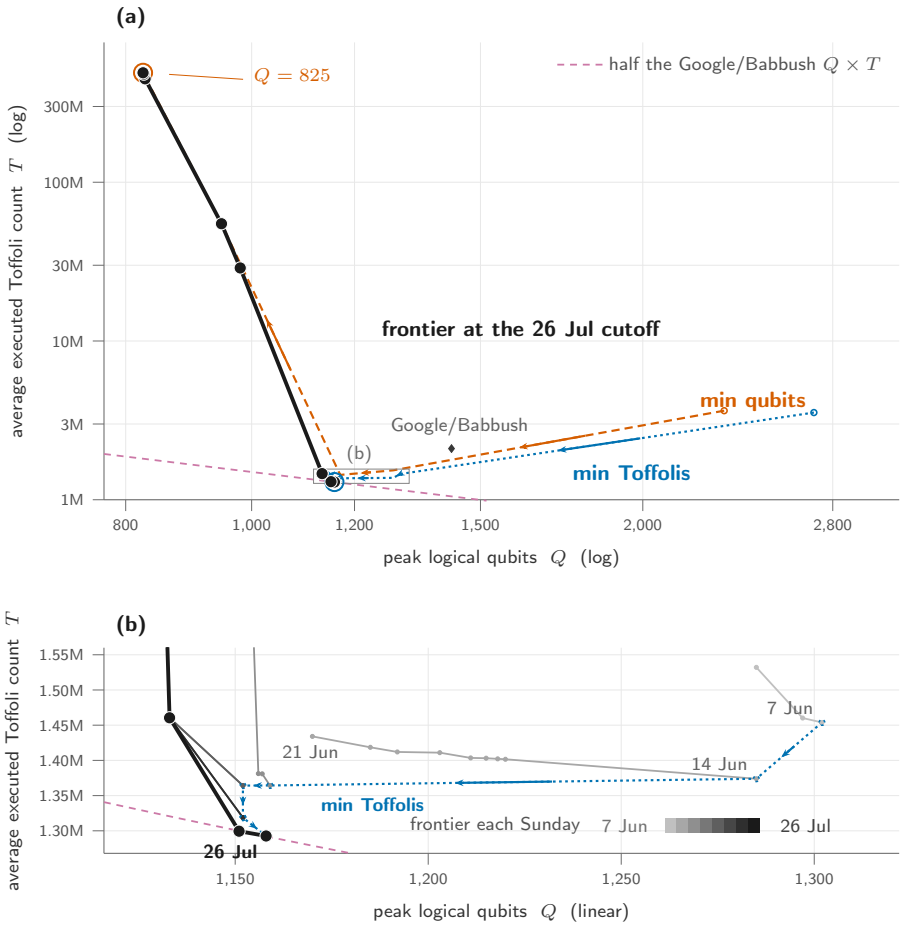


Fig. 6: Evolution of the resource frontier. **(a)** The Pareto frontier at the 26 July data cutoff in the (Q, T) resource plane, with peak logical qubits Q on the horizontal axis, average executed Toffoli count T on the vertical axis, and both axes shown on logarithmic scales. The thick black line and filled circles mark the admitted nondominated operating points at the cutoff, where *admitted* means that the circuit passes all 9,024 Fiat–Shamir-derived validation inputs. The orange dashed and blue dotted trajectories trace the minimum-width ("min qubits") and minimum-Toffoli ("min Toffolis") records over time; arrows indicate temporal progression rather than circuit ancestry. The diamond marks the reported Google/Babbush low-gate operating point [3], and the magenta dashed line is the constant-$Q \times T$ locus equal to one half of its reported product, crossed by the frontier at submission `8e9c9a2` with 1,151 qubits and 1,299,453 Toffoli gates. **(b)** A magnified view, on linear axes, of the low-Toffoli region enclosed by the gray box labeled (b) in panel (a). Gray polylines show the reconstructed Pareto frontier at successive Sunday checkpoints before the cutoff, with darker shades denoting later dates; the thick black line is the frontier at the cutoff. The blue dotted trajectory and arrows show the evolution of the minimum-Toffoli record. Several later weekly frontiers nearly coincide in this region, while the minimum-width trajectory leaves the displayed range after 21 June. Lines connecting operating points are guides to the eye.

1,151 and 1,299,453, respectively, lowering Q by 57.6%, T by 67.2%, and $Q \times T$ by 86.1%, from 10.75B to 1.496B, a 7.19-fold improvement. A separate low-width branch reached $Q = 825$ at approximately 489M Toffoli gates. Figure 6 visualizes the resulting frontier and its evolution over time, while Table 2 lists the eight admitted Pareto points at the cutoff.

The frontier contains two promoted points and six admitted low-width circuits preserved in public branches and submission notes. Here *admitted* means that a circuit passes all 9,024 Fiat–Shamir-derived validation inputs. The reported T is the challenge's average executed-Toffoli metric, not a static gate count, T-depth, or fault-tolerant hardware cost.

The cost of reducing width varies sharply across the frontier: moving from $Q = 828$ to $Q = 826$ adds 31.23M Toffoli gates, whereas moving from $Q = 1{,}158$ to $Q = 1{,}151$ adds only 6,802. These differences are descriptive secants between distinct circuit routes, not causal marginal costs. In particular, the product-efficient branch uses dialog-GCD record and replay, whereas the sub-1,000-qubit branch uses register-shared shrunken-PZ inversion. The sparsely populated region between them may therefore reflect this architectural discontinuity rather than a fundamental width boundary; a hybrid or otherwise undiscovered construction could smooth the apparent transition. Additional operating points and fixed-budget comparisons appear in Supplemental Note A.

5.1.2 Comparison with Prior Point-Addition Results Table 3 compares representative operating points from Babbush et al., Schrottenloher, TrailMix, and ECDSA.Fail, including their interfaces, correctness evidence, and windowing support [3, 59, 64]; Figure 2 places them alongside the challenge frontier. Among the window-supporting entries, the ECDSA.Fail *windowed* `8e9c9a2` circuit has the lowest tabulated retry-adjusted proxy, $Q \times T/\hat{p} \approx 1.961\,\mathrm{B}$, while providing a coherent single-call window-indexed point-addition interface. External rows without common-corpus measurements are evaluated optimistically with $\hat{p} = 1$.

The mixed-addition challenge incumbent is also numerically below the 1,175-qubit Babbush et al. point and the TrailMix low-qubit route on both Q and T axes, while the low-width branch reaches $Q = 825$ at approximately 489M executed Toffoli gates. These comparisons provide numerical context rather than establish formal dominance because interfaces, correctness assumptions, and accounting conventions differ; in particular, the coherent challenge construction is a point-addition kernel rather than a complete implementation of Shor's algorithm. The challenge additionally provides a public source history connecting each operating point to its preceding branches, optimization mechanisms, and local correctness obligations.

5.2 Best ECDSA.Fail Point-Addition Circuits at the Data Cutoff

This subsection describes the architectures and principal optimization mechanisms of the best ECDSA.Fail point-addition circuits at the data cutoff. We present the best $Q \times T$-scoring circuit and the best low-qubit construction in

Artifact	Interface	Reported Q	Reported T	Reported Error rate	Retry-adjusted $Q \times T/\hat{p}$	Window support
Google/Babbush low-Q [3]	Window-indexed	$\leq$ 1,175	$\leq$ 2.7 M	0/9,024 FS	$\approx$ 3.17 B*	Yes
Google/Babbush low-gate [3]	Window-indexed	$\leq$ 1,425	$\leq$ 2.1 M	0/9,024 FS	$\approx$ 2.99 B*	Yes
Schrottenloher space-optimized [59]	Window-indexed	1,208†	2.59 M†	Reported $\epsilon \leq 2^{-13.3}$; 10,000 tests	$\approx$ 3.13 B*	Yes
Schrottenloher gate-optimized [59]	Window-indexed	1,462†	2.06 M†	Reported $\epsilon \leq 2^{-13.3}$; 10,000 tests	$\approx$ 3.01 B*	Yes
TrailMix `jump-lowqubit` [64]	Mixed; classical addend	1,169	$\approx$ 2.09 M	0/9,000 FS	$\approx$ 2.44 B*	Not demonstrated
TrailMix `shrunken-PZ` [64]	Mixed; classical addend	1,050	$\approx$ 32.3 M	0/9,000 FS	$\approx$ 33.9 B*	Not demonstrated
ECDSA.Fail `8e9c9a2`	Mixed; classical addend	1,151	1.30 M	0/9,024 FS; 0.192% on 100,000 independent cases	$\approx$ 1.499 B	No
ECDSA.Fail windowed `8e9c9a2`	Window-indexed	1,162	1.68 M	0.191% on 100,000 independent cases	$\approx$ 1.961 B	Yes
ECDSA.Fail `b6f2b0a`	Mixed; classical addend	825	$\approx$ 489 M	0/9,024 FS	$\approx$ 403 B*	Not demonstrated
ECDSA.Fail `3616dbf` (post-cutoff)	Mixed; classical addend	1,321	$\approx$ 0.953 M	0/9,024 FS	$\approx$ 1.259 B*	Not demonstrated

Table 3: Reported resources, correctness evidence, and retry-adjusted spacetime-resource proxies for representative `secp256k1` point-addition circuits. Here B $= 10^9$. Under an independently rerunnable classical-success model, a circuit with empirical success probability $\hat{p}$ requires $1/\hat{p}$ executions on average to obtain one successful result. Accordingly, $Q \times T/\hat{p}$ provides a sensitivity-adjusted proxy when success probabilities and failure events are defined and measured comparably. Among the window-supporting circuits listed in the table, the ECDSA.Fail windowed `8e9c9a2` construction has the lowest tabulated $Q \times T/\hat{p}$, at approximately 1.961 B, while supporting the required single-call coherent window-indexed point-addition interface. To avoid penalizing external circuits for which no common-corpus estimate is available, rows marked * conservatively assume zero error, or $\hat{p} = 1$; these are therefore optimistic estimates, and any nonzero error would increase the required work. The Google/Babbush entries remain upper bounds because their published Q and T values are themselves upper thresholds. FS denotes finite Fiat–Shamir support, whereas "independent" denotes a corpus selected independently of the circuit hash. Failure definitions and accounting conventions differ across sources, so the comparison remains contextual. Window support denotes a coherent window-indexed point-addition interface, not a complete implementation of Shor's algorithm. †The Schrottenloher values include the separately reported $w = 16$ overhead of 16 qubits and $3 \cdot 2^{16}$ Toffoli gates; the corresponding arithmetic-core values are $(1{,}192, 2.39\,\text{M})$ and $(1{,}446, 1.86\,\text{M})$.

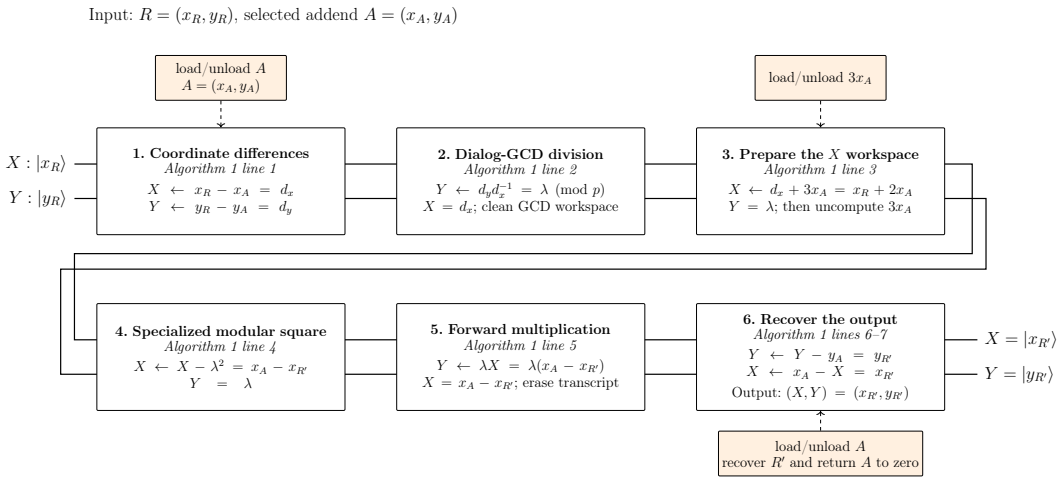


Fig. 7: High-level architecture of the best $Q \times T$-scoring `8e9c9a2` mixed point-addition circuit. The six phases implement the in-place map $R \mapsto R' = R + A$ while recycling the X and Y coordinate registers. Under the challenge evaluator, the circuit uses $Q = 1{,}151$ logical qubits and $T = 1{,}299{,}453$ average executed Toffoli gates. The lookup/unlookup boxes indicate where a fully windowed Shor construction would temporarily materialize the selected addend; in the challenge benchmark, (x_A, y_A) are provided as classical inputs.

Section 5.2.1 and Section 5.2.2, emphasizing their distinct resource objectives and architectural trade-offs. The key circuit optimization techniques underlying these circuits are provided in Section 5.3.

5.2.1 Best $Q \times T$-Scoring Circuit As of the data cutoff, the promoted submission with the best $Q \times T$ score was `8e9c9a2`, with $Q = 1{,}151$, $T = 1{,}299{,}453$, and $Q \times T = 1{,}495{,}670{,}403$. Its promoted source commit is `60d61859fa69`. We place the cutoff date at 26 July 2026, 09:21:55 UTC, when that submission's API record was last updated. This subsection provides an overview of the circuit, where a more comprehensive analysis of its arithmetic components and principal optimizations is available in Supplemental Note D: Technical Anatomy of Submission 8e9c9a2.

Figure 7 presents the six-stage in-place computation of $R' = R + A$, where $R = (x_R, y_R)$ is the quantum accumulator and $A = (x_A, y_A)$ is the selected addend. Stage 1 forms $d_x = x_R - x_A$ and $d_y = y_R - y_A$, and Stage 2 computes the slope $\lambda = d_y d_x^{-1} \bmod p$. Stage 3 updates the recycled X register to $d_x + 3x_A = x_R + 2x_A$, after which Stage 4 subtracts λ^2 to obtain $X = x_A - x_{R'}$. Stage 5 computes $Y = \lambda(x_A - x_{R'})$, and Stage 6 recovers $y_{R'} = Y - y_A$ and $x_{R'} = x_A - X$.

The same X and Y registers are therefore reused throughout, avoiding separate output-coordinate registers.

Stages 2 and 5 use dialog-GCD, the record-and-replay Euclidean architecture introduced by Khattar et al. [33] and subsequently adapted to ECDLP point addition by Schrottenloher [59]. Its forward Euclidean traversal evolves a shrinking operand pair (u, v) and records the parity, comparison, swap, subtraction, and halving decisions needed for reversal. The Jump-2 GCD groups consecutive Euclidean operations into larger steps (Section 5.3.1), while the base-5 codec exploits the five reachable control symbols to encode three steps in seven qubits rather than nine (Section 5.3.2). Reverse replay decodes the transcript to update the Bézout registers, restore the Euclidean operand, and clear the transcript. The resulting reversible map implements $Y \mapsto YX^{-1}$ in Stage 2 and, when used in the opposite computational direction, $Y \mapsto YX$ in Stage 5. Storage released as (u, v) shrink is reused for the growing transcript and temporary arithmetic state, reducing peak width.

The remaining stages use complementary optimizations. Stages 1 and 6 serialize the materialization of classical coordinates and employ truncated pseudo-Mersenne correction and measurement-assisted carry cleanup. Stage 3 materializes the classically computed value $3x_A \bmod p$ only for the required addition and immediately unloads it (Section 5.3.6). Stage 4 uses streamed, symmetry-aware Karatsuba squaring together with $2^{256} \equiv 2^{32} + 977 \pmod{p}$ (Section 5.3.4). Constant propagation and post-generation dead-code and redundancy elimination apply across the complete circuit (Sections 5.3.6 and 5.3.7), with additional optimizations documented in Supplemental Note B: List of agent-identified optimizations. Finally, the QROM boundaries in Figure 7 indicate the lookup/use/unlookup interface required for integration with windowed Shor, which we implement and evaluate for this circuit in Section 5.2.3.

The circuit with the best $Q \times T$ score at the data cutoff can be retrieved and evaluated using the artifact and pinned-repository procedure in Section 5.5.

5.2.2 Best low-Q Circuit As of the data cutoff, the selected low-Q endpoint was submission `b6f2b0a`, with $Q = 825$, $T = 489{,}161{,}900$, and $Q \times T = 403{,}558{,}567{,}500$. This operating point explores the extreme low-width region of the reversible time–space trade-off: it uses approximately 28% fewer qubits than the best $Q \times T$ circuit, but substantially more executed Toffoli gates. A subsequent public submission, `b4a51d2`, reported an 813-qubit circuit. A comprehensive source-grounded analysis of the arithmetic stages, live-register allocation, and principal optimizations of the 825-qubit circuit is available in Supplemental Note E: Technical Anatomy of Submission b6f2b0a.

The circuit computes the in-place mixed addition $R' = R + A$ using the six stages shown in Figure 8. We retain the paper-wide definitions $d_x = x_R - x_A$ and $d_y = y_R - y_A$, while the low-Q implementation stores their negations, $\widetilde{d}_x = x_A - x_R = -d_x$ and $\widetilde{d}_y = y_A - y_R = -d_y$, in the live registers X and Y. Reversing both signs leaves the slope unchanged: $\lambda = \widetilde{d}_y \widetilde{d}_x^{-1} = (-d_y)(-d_x)^{-1} = d_y d_x^{-1} \pmod{p}$.

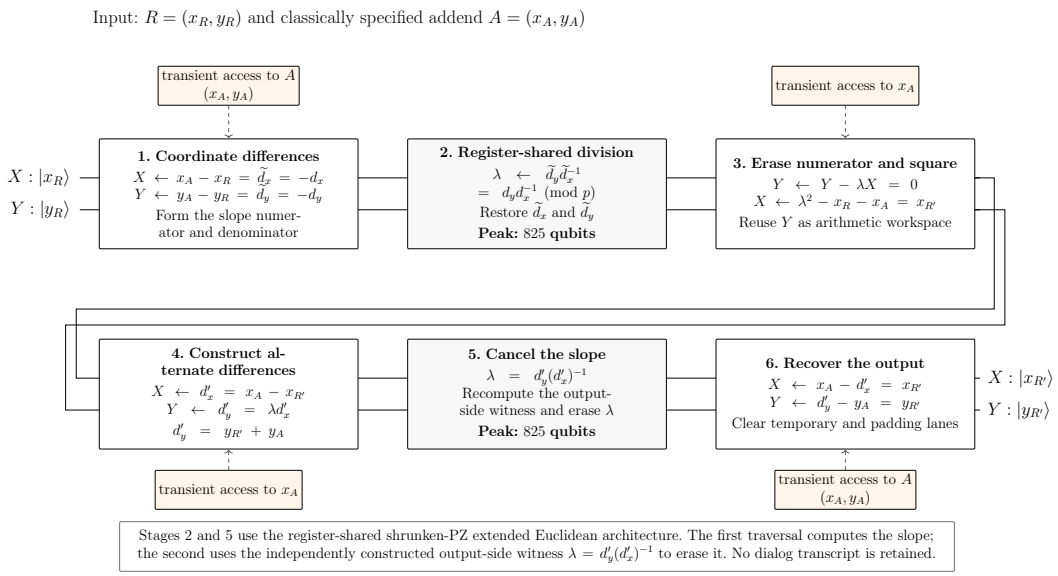


Fig. 8: High-level architecture of the low-Q `b6f2b0a` mixed point-addition circuit. The paper-wide differences are $d_x = x_R - x_A$ and $d_y = y_R - y_A$, whereas the implementation stores their negations $\widetilde{d}_x = -d_x$ and $\widetilde{d}_y = -d_y$; their ratio therefore yields the same slope. The six stages implement $R \mapsto R' = R + A$ while recycling the X and Y coordinate registers. Register-shared extended-Euclidean computations in Stages 2 and 5 determine the peak width. Under the challenge evaluator, the circuit uses $Q = 825$ logical qubits and $T = 489{,}161{,}900$ average executed Toffoli gates. The dashed arrows indicate transient access to the classically specified addend; integration with windowed Shor requires a separately implemented and validated QROM lookup/use/unlookup interface.

Stage 1 forms $X = \widetilde{d}_x$ and $Y = \widetilde{d}_y$, and Stage 2 computes $\lambda = \widetilde{d}_y \widetilde{d}_x^{-1} = d_y d_x^{-1} \bmod p$ while reversibly restoring the source-level differences needed for cleanup. Stage 3 erases the stored numerator through $Y \leftarrow Y - \lambda X = \widetilde{d}_y - \lambda \widetilde{d}_x = -d_y + \lambda d_x = 0$ and computes $x_{R'} = \lambda^2 - x_R - x_A$, allowing the released Y register to serve as arithmetic workspace. Stage 4 constructs the output-side witnesses $d'_x = x_A - x_{R'}$ and $d'_y = \lambda d'_x = y_{R'} + y_A$. Hence $\lambda = d_y d_x^{-1} = d'_y (d'_x)^{-1} \pmod p$. Stage 5 uses the second identity to erase the slope through a fresh reversible division, without retaining the inversion history from Stage 2. Finally, Stage 6 recovers $x_{R'} = x_A - d'_x$ and $y_{R'} = d'_y - y_A$ in the original coordinate registers. The two inversion stages determine the 825-qubit peak; the remaining coordinate-arithmetic stages remain below it.

Unlike the best $Q \times T$ circuit, this design computes $\lambda = d_y d_x^{-1} = \widetilde{d}_y \widetilde{d}_x^{-1} \bmod p$ without dialog-GCD or a compressed transcript retained for later replay. Instead, it descends from TrailMix's `shrunken-PZ` architecture [64], which implements a Proos–Zalka-style extended Euclidean algorithm as a fixed reversible state

machine [52]. The shrinking Euclidean remainders, growing cofactors, quotient bits, and associated length metadata share two 259-qubit physical banks whose ownership boundaries move during the computation. During each inversion peak, these banks coexist with a 257-qubit passenger coordinate and 37 persistent metadata qubits:

$$\underbrace{259 + 259 + 257 + 37}_{\text{812 persistent qubits}} + \underbrace{13}_{\text{maximum local workspace}} = 825. \tag{15}$$

Step-dependent widths, dynamic cursors, and reversible borrowing allow inactive high-order lanes to host quotient, comparison, carry, and control information, provided that each lane is restored before its logical owner becomes active again. The underlying inversion algorithm, packing invariants, and register-sharing schedule are detailed in Section 5.3.5.

The reduction from TrailMix's reported 1,050-qubit construction to the first 980-qubit challenge circuit retained the same shrunken-PZ inversion architecture but tightened its live-information schedule. The principal savings came from support-calibrated, step-dependent bounds for the Euclidean state, narrower shift and quotient metadata, and removal of a separate loaded-coordinate register at competing arithmetic peaks. This 70-qubit reduction was therefore primarily a scheduling and liveness improvement within the inherited architecture, rather than the introduction of the register-shared EEA described below.

A subsequent architectural transition reduced the circuit from 947 to 851 qubits by reimplementing the shrunken-PZ extended Euclidean computation with substantially more aggressive register sharing. Shrinking remainders, growing Bézout coefficients, and temporary quotient state were packed into two shared physical work registers, with dynamic cursor management and movable ownership boundaries used to address the active subranges. This construction, described in Section 5.3.5, established the architecture from which the later 851-to-825 reductions descended. Those reductions added denser lane hosting, measurement-assisted release and recomputation, narrower metadata, and increasingly local scratch schedules. The final 826-to-825 transition removed one persistent quotient-length bit and reconstructed its omitted high-bit effect only where it was consumed.

The algebraic point-addition identities and reversible cleanup steps are exact under the generic-addition preconditions. Some step-dependent width and metadata bounds are calibrated to the challenge's finite validation support and should therefore be interpreted as validated benchmark envelopes rather than all-input width theorems. As with the best $Q \times T$ circuit, the benchmark supplies the addend A classically; integration into windowed Shor requires a separately implemented and validated QROM lookup/use/unlookup interface.

The circuit can be retrieved and evaluated using the artifact and pinned-repository procedure in Section 5.5.

5.2.3 Windowed-Addition Compatibility of the Best-Scoring Circuit

The benchmark supplies the addend A classically, whereas windowed Shor arith-

metic selects it coherently from a precomputed table using a quantum address (Sections 2.2 and 3.1). To measure the resulting interface overhead, we constructed a windowed-compatible variant of `8e9c9a2`. It exposes the table as $2 \cdot 2^w$ classical coordinate inputs alongside the quantum coordinate registers X, Y and a w-qubit address register J. Every table-dependent operation is conditioned on these input bits, making the circuit topology independent of the table contents; shifted G- or P-tables can therefore be substituted without recompilation. Each of the three stages that uses A loads its selected coordinates through QROM immediately before use and uncomputes them afterward, so the selected point is not live during the dialog-GCD or multiplication peaks. Derived values such as $3x_A$ are computed within the circuit rather than stored in additional table columns, while the identity row $A = \mathcal{O}$ is handled coherently by conditioning the arithmetic on a nonzero-address predicate.

Because each arithmetic stage preserves the loaded payload until it is no longer needed, reversing the QROM traversal returns its payload registers exactly to zero. Routing ancillas are then removed by measurement-based uncomputation with a conditional phase correction, leaving no address-dependent payload, ancilla, or selection phase. The resulting lookup/use/unlookup map is therefore coherent by linearity and exact reversible replay. Although the evaluator simulates only computational-basis addresses, we tested individual addresses and correlated four-call sequences for every $w \in \{0, \ldots, 5\}$, confirming workspace cleanup after each call.

For $w = 16$, the resulting circuit uses $Q = 1{,}162$ logical qubits and $T = 1{,}684{,}161$ average executed Toffoli gates on the same 100,000-input corpus used for the baseline. Relative to the original `8e9c9a2`, this adds 11 qubits (0.96%) and approximately 29.6% average executed Toffoli per addition. Its any-channel error rate was 0.191%, versus 0.192% for the baseline on the same independently seeded 100,000-input corpus. The paired comparison had 29 windowed-only and 30 baseline-only failures (two-sided exact sign test, $p = 1.0$), providing no detectable difference on this corpus; no additional classical-output failures were attributable to coherent selection.

These counts follow the challenge's $Q \times T$ convention and exclude the fault-tolerant costs of table-payload CNOTs, classical table storage, measurements, and feed-forward. Those operations increase the static operation stream by a factor of 23.36 and may be significant under other architectures. The construction provides a reusable window-indexed point-addition kernel that coherently handles the identity-table row $A = \mathcal{O}$ by bypassing the arithmetic. For $A \neq \mathcal{O}$, the construction inherits the source circuit's approximate arithmetic, lack of a coherent failure flag, and generic-affine domain from Section 3; in particular, it requires $R \neq \mathcal{O}$ and $R \notin \{A, -A, -2A\}$. Instantiating every shifted table and integrating the complete window schedule, Fourier transforms, and classical postprocessing also remain future work. Nevertheless, the result shows that the benchmark's classically supplied addend is an interface choice rather than a fundamental capability gap, with the compatible variant remaining in the low-qubit, low-Toffoli region of Table 3. Supplemental Note F provides the complete con-

struction, resource breakdown, and validation; the implementation is available in the source repository.

5.3 Principal Circuit Optimization Techniques

This subsection examines the principal techniques underlying the circuits discussed in the previous subsection, including the Jump-2 Euclidean algorithm (Section 5.3.1), base-5 transcript encoding (Section 5.3.2), specialized squaring (Section 5.3.4), register-shared extended Euclidean inversion (Section 5.3.5), constant propagation (Section 5.3.6), and dead-code and redundancy elimination (Section 5.3.7). We additionally document the post-cutoff comparison-free ping-pong dialog-GCD architecture (Section 5.3.3), which replaces rather than extends the earlier Jump-2/base-5 backend.

5.3.1 The Jump-2 Euclidean Algorithm The baseline dialog-GCD record phase in Algorithm 2 records one symbol $\sigma_i = (b_i, s_i)$ per ordinary divstep, where b_i indicates subtraction and s_i indicates a preceding swap. Jump-2 preserves the resulting Euclidean transformation but retimes consecutive divsteps into larger macro-steps. The division that necessarily follows a subtraction of two odd operands is moved to the beginning of the next macro-step; if the intermediate value remains even, a second division is performed before the next possible subtraction.

At every normal macro-step boundary, v is even. The first division is therefore unconditional, and the macro-step realizes

$$v \longmapsto \begin{cases} v/2, & v/2 \text{ is odd}, \\ v/4, & v/2 \text{ is even}. \end{cases} \tag{16}$$

It then performs the next required comparison, swap, and subtraction. The ordinary symbol (b_i, s_i) is extended to $\sigma_i = (b_i, s_i, s_{2,i})$, where $s_{2,i}$ records whether the second division in (16) occurred.

Every macro-step again leaves v even. If $b_i = 0$, the value remains even after the divisions; if $b_i = 1$, both active operands are odd and their difference is even. The next normal macro-step may therefore begin with its unconditional division. The initial macro-step is exceptional because the parity of x is unknown, so t_1 records whether its first division occurs. Jump-2 is an exact retiming of the ordinary dialog-GCD path: it reduces the number of macro-step boundaries, comparisons, swaps, and subtraction bodies without changing the induced arithmetic transformation.

Use in dialog-GCD. For the transcript generated by a nonzero operand x, let D_x denote the induced Euclidean transformation. Once the transcript is fixed, D_x is a linear map over $\mathbb{F}_p$ because it consists only of swaps, additions or subtractions, and multiplication by powers of 2^{-1}. Successful convergence gives $D_x(p, x) = (1, 0)$, which reduces modulo p to $D_x(0, x) = (1, 0)$. Since $x \neq 0$, its inverse x^{-1}

Algorithm 4 Jump-2 Euclidean macro-steps

Input: $(u, v) = (p, x)$ with $x \in \mathbb{F}_p^*$ and odd u; fixed macro-step budget L
Output: transcript $\tau = (\sigma_0, \ldots, \sigma_{L-1})$

for $i = 0, \ldots, L-1$ **do**
  if $i = 0$ **then**
    $t_1 \leftarrow [v \text{ is even}]$; $v \leftarrow v/2$ if $t_1 = 1$
  else
    $v \leftarrow v/2$ ▷ unconditional at a normal boundary
  end if
  $s_{2,i} \leftarrow [v \text{ is even}]$; $v \leftarrow v/2$ if $s_{2,i} = 1$
  $b_i \leftarrow [v \text{ is odd}]$ ▷ subtraction flag
  $s_i \leftarrow b_i$ if $i = 0$, otherwise $s_i \leftarrow b_i \wedge [v < u]$ ▷ swap flag
  $(u, v) \leftarrow (v, u)$ if $s_i = 1$
  $v \leftarrow v - u$ if $b_i = 1$
  append $\sigma_i = (b_i, s_i, s_{2,i})$ to τ ▷ step 0 also records t_1
end for

exists in $\mathbb{F}_p$ and any $y \in \mathbb{F}_p$ can be written as $y = (yx^{-1})x$. Linearity allows the scalar yx^{-1} to be pulled through D_x, giving $D_x(0, y) = D_x\big(0, (yx^{-1})x\big) = yx^{-1}D_x(0, x) = (yx^{-1}, 0)$, which is the division map in Equation (17). Likewise, scaling by y gives $D_x(0, yx) = (y, 0)$, and applying D_x^{-1} gives the corresponding multiplication map:

$$D_x(0, y) = (yx^{-1}, 0), \qquad D_x^{-1}(y, 0) = (0, yx). \tag{17}$$

Stage 2 uses the first map with $x = d_x$ and $y = d_y$ to obtain $\lambda = d_y d_x^{-1}$, while Stage 5 uses the inverse map with the then-current X operand to perform in-place multiplication.

During reverse replay, each symbol controls the inverse macro-step

$$\begin{aligned} &\text{if } b_i: \quad z \leftarrow z + y \pmod p, \\ &\text{if } s_i: \quad (y, z) \leftarrow (z, y), \\ &z \leftarrow 2^{k_i} z \pmod p, \end{aligned} \qquad k_i = \begin{cases} t_1 + s_{2,0}, & i = 0, \\ 1 + s_{2,i}, & i > 0. \end{cases} \tag{18}$$

Modular addition reverses subtraction, swapping reverses itself, and multiplication by 2^{k_i} reverses the corresponding divisions because p is odd. Each decoded symbol is used to update the payload and reverse its Euclidean macro-step before being cleared. The doubling or quadrupling operation shares a pseudo-Mersenne correction using $2^{256} \equiv 2^{32} + 977 \pmod p$, while the transcript encoding and streamed replay schedule are described in Section 5.3.2.

A reversible circuit cannot terminate after an input-dependent number of macro-steps, so the implementation uses a fixed budget. Increasing it from 258 to 261 macro-steps reduced the observed nonconvergence frequency from 2.839×10^{-4} to 2.86×10^{-5} over ten million inputs. Conditional on convergence, Jump-2 exactly reproduces the baseline dialog-GCD transformation; approximation arises only from the finite schedule or separately documented width truncations.

5.3.2 Base-5 Codec for the Jump-2 Transcript Because the Euclidean operands may be in superposition, the Jump-2 symbols can be entangled with the input and must be stored coherently in the transcript τ. A raw symbol $\sigma_i = (b_i, s_i, s_{2,i})$ occupies three qubits, but the update rules permit only five of the eight Boolean triples. If $s_{2,i} = 0$, the value after the first division is odd and therefore $b_i = 1$; moreover, a swap requires a subtraction, so $s_i = 1$ implies $b_i = 1$. The reachable alphabet is

$$\mathcal{S} = \{(0,0,1), (1,0,0), (1,1,0), (1,0,1), (1,1,1)\}. \tag{19}$$

A fixed bijection $\phi : \mathcal{S} \to \{0,1,2,3,4\}$ therefore represents each symbol as a base-5 digit $d_i = \phi(\sigma_i)$.

Three consecutive digits can be packed into the integer

$$z = d_0 + 5d_1 + 25d_2, \qquad 0 \le z < 5^3 = 125 < 2^7. \tag{20}$$

Thus, although three raw symbols occupy nine qubits, their $5^3 = 125$ reachable combinations fit in a seven-qubit register; similarly, two symbols have $5^2 = 25$ combinations and require five rather than six qubits. Writing $|a\rangle_m$ for the m-qubit computational-basis encoding of a, the reversible encoder acts on the reachable support as

$$|\sigma_0\rangle_3 |\sigma_1\rangle_3 |\sigma_2\rangle_3 \longmapsto |z\rangle_7 |0\rangle^{\otimes 2}, \qquad z = d_0 + 5d_1 + 25d_2. \tag{21}$$

The encoder therefore does not discard two qubits; it returns them coherently to $|0\rangle$ so they can be released or reused. Its inverse reconstructs all three symbols exactly. To define a valid reversible circuit on the complete nine-qubit Hilbert space, the implementation extends this action to a permutation of all computational-basis states, including those outside the reachable support $\mathcal{S}^3$.

The initial macro-step is encoded separately because it also contains t_1; its four reachable states fit in two qubits. Of the remaining 260 ordinary symbols, 258 are stored in 86 seven-qubit triple blocks. The final two symbols form an incomplete group and are packed into one five-qubit pair block, since $5^2 = 25 < 2^5$. The complete 261-step layout therefore uses

$$2 + 86 \cdot 7 + 5 = 609 \tag{22}$$

qubits, compared with $261 \cdot 3 = 783$ qubits for raw storage, reducing the transcript by 174 qubits.

Encoding and replay are streamed rather than performed only after the complete walk. As the Euclidean operands shrink, completed symbol groups are encoded into storage released by their inactive high-order bits. Reverse replay decodes one group at a time, applies its symbols to the payload and Euclidean state, and returns both the decoded controls and encoded block to zero. The codec therefore changes neither the Euclidean path nor the resulting multiplication or division; it exchanges reversible encoding work for a smaller persistent transcript and lower peak width.

5.3.3 Comparison-Free Ping-Pong Dialog-GCD After the paper's data cutoff, submission `3616dbf` replaced the Jump-2/base-5 backend with a different dialog-GCD architecture. An ordinary binary-GCD step compares two full-width quantum integers and may then swap them. A reversible comparison examines many bits, produces a control that must later be cleared, and controls a wide swap and subtraction. The ping-pong construction instead chooses addition or subtraction from only the two lowest bits and updates the operand registers in a fixed alternating order. It therefore removes full-width magnitude comparisons and data-dependent swaps from the Euclidean value walk. Here, "comparison-free" refers only to that walk; bounded carry and overflow tests remain inside the modular replay cells.

The construction combines two established ideas in a new circuit organization. Low-bit plus-minus GCD methods replace magnitude comparisons with tests modulo 4 [7], while dialog-GCD records a Euclidean path for later coefficient replay [33, 59]. The challenge-specific specialization keeps both signed operands odd, alternates the updated operand deterministically, records one sign bit per round, and implements division and multiplication through fused replay cells.

Motivating example. We illustrate how the ping-pong value walk and its transcript are used to obtain the inverse of $x = 5$ modulo $p = 13$. Starting from the odd pair $(13, 5)$, the method alternately updates the second and first entries. In each round, the entry being updated is called the *target*, and the other entry is called the *source*. If the two entries have the same residue modulo 4, the target is replaced by half their sum; otherwise, it is replaced by half the difference between the target and the source:

$$\begin{aligned}(13,5) \xrightarrow{+} (13,9) \xrightarrow{+} (11,9) \xrightarrow{-} (11,-1) \xrightarrow{+} (5,-1)\\ \xrightarrow{-} (5,-3) \xrightarrow{+} (1,-3) \xrightarrow{+} (1,-1).\end{aligned} \tag{23}$$

The target alternates at every arrow. For example, the first round updates the second entry as $(5 + 13)/2 = 9$, while the third updates it as $(9 - 11)/2 = -1$. No round compares operand magnitudes or performs a data-dependent swap. The one-bit transcript is $\tau = (0, 0, 1, 0, 1, 0, 0)$, where 0 denotes addition and 1 denotes subtraction. Applying the transformations selected by τ modulo 13 to a coefficient pair initialized as $(0, 1)$ yields $(8, 5) \equiv (8, -8) \pmod{13}$, where division by two is implemented as multiplication by $2^{-1} = 7 \pmod{13}$. Correcting the negative terminal sign gives $5^{-1} = 8 \pmod{13}$, consistent with $5 \cdot 8 \equiv 1 \pmod{13}$.

Odd-preserving update and correctness. For a general nonzero denominator $x \in \mathbb{F}_p$, the circuit begins with two odd operands by choosing

$$x^\star = \begin{cases} x, & x \text{ is odd}, \\ x - p, & x \text{ is even}, \end{cases} \qquad x^\star \equiv x \pmod{p}. \tag{24}$$

Algorithm 5 Comparison-free ping-pong value walk

Input: nonzero denominator $x \in \mathbb{F}_p$; fixed round budget L
Output: terminal signed pair and transcript τ

$\rho_0 \leftarrow p$; $\rho_1 \leftarrow x$ if x is odd, otherwise $\rho_1 \leftarrow x - p$
$\tau \leftarrow ()$
for $i = 0, \ldots, L-1$ **do**
  if i is even **then**
    $(s, t) \leftarrow (\rho_0, \rho_1)$ ▷ update ρ_1
  else
    $(s, t) \leftarrow (\rho_1, \rho_0)$ ▷ update ρ_0
  end if
  $e_i \leftarrow t[1] \oplus s[1]$
  $t \leftarrow (t + (-1)^{e_i} s)/2$
  append e_i to τ
end for
Successful endpoint: $(\rho_0, \rho_1) = (\eta_0, \eta_1)$ with $\eta_0, \eta_1 \in \{-1, +1\}$

Since p is odd, both p and $x^\star$ are odd. In each round, let s be the source and t the predetermined target. For an odd signed integer z, let $z[1]$ distinguish $z \equiv 1 \pmod 4$ from $z \equiv 3 \pmod 4$. The circuit records

$$e = t[1] \oplus s[1], \qquad t' = \frac{t + (-1)^e s}{2}. \tag{25}$$

If the low-bit patterns agree, the sum is $2 \pmod 4$; if they differ, the target-minus-source difference is $2 \pmod 4$. Thus, the numerator is divisible by two but not by four, so t' is again odd. Moreover, $\gcd(s, t) = \gcd(s, t \pm s) = \gcd(s, (t \pm s)/2)$ because the common divisor of two odd operands is odd. Every round therefore preserves the GCD. The round is also reversible once e is known, since $t = 2t' - (-1)^e s$. Consequently, the transcript uniquely determines both the forward transformation and its inverse. Since $\gcd(p, x^\star) = 1$, reaching a signed-unit pair certifies successful convergence.

The implementation uses $L = 704$ rounds and stores one transcript qubit per round. This is wider than the 609-qubit base-5 Jump-2 transcript, but its one-bit symbols require cheaper replay. Signed-unit pairs are stable under subsequent rounds, so reversible padding preserves an endpoint reached before the fixed schedule ends.

Dialog replay. For the transcript generated by x, let D_x denote the resulting linear transformation. On a converged path,

$$D_x(p, x^\star) = (\eta_0, \eta_1), \qquad \eta_0, \eta_1 \in \{-1, +1\}. \tag{26}$$

Modulo p, $(p, x^\star)$ is equivalent to $(0, x)$. Since every transcript-selected update is linear over $\mathbb{F}_p$, applying the same transformation to $(0, y)$ gives

$$D_x(0, y) = yx^{-1}(\eta_0, \eta_1) \pmod p. \tag{27}$$

After correcting the terminal signs, both coefficient registers contain yx^{-1}; one copy is retained and the other is cleared. Stage 2 applies this map with $x = d_x$ and $y = d_y$ to obtain $\lambda = d_y d_x^{-1}$.

Multiplication uses the inverse round derived from Equation (25). If $c'_t = (c_t + (-1)^{e_i} c_s)/2$, then

$$c_t = 2c'_t - (-1)^{e_i} c_s \pmod{p}. \tag{28}$$

Reverse replay therefore maps the sign-corrected terminal pair to $(0, yx)$. Stage 5 uses this direction with $x = x_A - x_{R'}$ and $y = \lambda$. The circuit then reverses the value walk, restores x, undoes the odd lift, and clears the transcript and workspace.

Circuit realization and resource effect. Each replay round is implemented as one fused modular cell. Division combines signed addition with modular halving, while multiplication combines modular doubling with the signed correction in Equation (28). For $p = 2^{256} - (2^{32} + 977)$, overflow at bit 256 becomes a sparse low-word correction. The implementation processes these corrections in short chunks and uses measurement-assisted cleanup for temporary products and carries.

Relative to its reproduced source parent, the ping-pong circuit increases peak width from 1,150 to 1,321 qubits but reduces average executed Toffoli count from 1,284,776 to 952,707, a reduction of 25.85%. Its $Q \times T$ score decreases from 1,477,492,400 to 1,258,525,947, or 14.82%. The construction therefore spends transcript width to obtain a simpler transition alphabet and cheaper replay.

5.3.4 Karatsuba Squaring with Pseudo-Mersenne Modulo The point-addition circuit requires the modular square-subtract transformation

$$|\lambda\rangle\,|X\rangle\,|0\rangle_W \longmapsto |\lambda\rangle\,\big|X - \lambda^2 \bmod p\big\rangle\,|0\rangle_W\,, \tag{29}$$

where W denotes temporary workspace and $p = 2^{256} - F$ with $F = 2^{32} + 977$. Modular squaring is a standard component of quantum elliptic-curve point addition [27, 54, 59], and the construction uses the established Karatsuba decomposition [50]. Its principal contribution is to specialize this decomposition to squaring, accumulate directly into the destination register, exploit the pseudo-Mersenne form of the `secp256k1` modulus, and schedule the computation reversibly without retaining a full-width product.

Let $\beta = 2^{128}$ and split the slope as $\lambda = L + \beta H$, where $0 \le L, H < 2^{128}$. Then $\lambda^2 = L^2 + 2\beta LH + \beta^2 H^2$. Define the three half-size squares $A = L^2$, $B = H^2$, and $C = (L+H)^2$. Since $C = A + 2LH + B$, the cross term is $2LH = C - A - B$, yielding the one-level Karatsuba decomposition

$$\lambda^2 = A + \beta(C - A - B) + \beta^2 B. \tag{30}$$

The construction further specializes Equation (30) to the `secp256k1` prime. Since $\beta^2 = 2^{256} \equiv F \pmod{p}$,

$$\lambda^2 \equiv A + \beta(C - A - B) + FB \pmod{p}. \tag{31}$$

The sparse signed-binary representation $F = 2^{32} + 2^{10} - 2^5 - 2^4 + 1$ allows multiplication by F to use fixed shifts and signed additions rather than a generic 256-bit modular multiplication or reduction.

The circuit does not materialize the reduced value λ^2 in a separate register. Instead, it applies the three contributions in Equation (31) directly to X:

$$C = (L + H)^2 : \qquad X \leftarrow X - \beta C, \tag{32}$$

$$A = L^2 : \qquad X \leftarrow X - A + \beta A, \tag{33}$$

$$B = H^2 : \qquad X \leftarrow X + \beta B - FB. \tag{34}$$

Their sum is $-\beta C - A + \beta A + \beta B - FB = -[A + \beta(C - A - B) + FB]$, which implements $X \leftarrow X - \lambda^2 \pmod{p}$.

The updates follow a build–apply–unbuild schedule. The circuit constructs C in clean workspace, applies $-\beta C$ to X, and reverses the squaring network to clear the workspace; it then repeats this sequence for A and B. The temporary register holding $L + H$ exists only while constructing C. Consequently, A, B, and C are never simultaneously live, and no persistent 512-bit product register is required.

Each half-size product is implemented as a symmetric square rather than a generic multiplication. For $z = \sum_i z_i 2^i$, symmetry gives $z^2 = \sum_i z_i 2^{2i} + 2\sum_{i<j} z_i z_j 2^{i+j}$. The diagonal terms satisfy $z_i^2 = z_i$, while each off-diagonal product $z_i z_j$ is computed only once and contributes with a one-bit shift. A direct 256-bit symmetric square contains $256 \cdot 255/2 = 32{,}640$ off-diagonal products, whereas the two 128-bit squares and one 129-bit sum-square used here contain $2\binom{128}{2} + \binom{129}{2} = 24{,}512$, approximately 24.9% fewer. Temporary product bits and arithmetic carries can be uncomputed as soon as their final contribution to X has been applied.

The construction therefore combines a square-specific Karatsuba decomposition, symmetry-aware partial-product generation, direct accumulation into the point-addition register, pseudo-Mersenne reduction, and phase-local reversible cleanup. The algebraic identity is exact; any approximation in a benchmark circuit arises only from separately documented finite-width modular-correction subroutines, not from the Karatsuba decomposition itself.

5.3.5 Register-Shared EEA and Lane-Lifetime Refinements The 947-to-851-qubit transition replaced the preceding shrunken-PZ inversion backend with a source-level implementation of the register-shared extended Euclidean algorithm (EEA) of Luo et al. [40]. The underlying principle, i.e. sharing storage between shrinking Euclidean remainders and growing Bézout coefficients, was introduced by Proos and Zalka [52]; Luo et al. supplied an explicit reversible construction using two shared work registers, embedded quotient storage, moving field boundaries, a rotation cursor, and step-dependent active arithmetic. The challenge contribution is an implementation and refinement of this architecture within the benchmark point-addition circuit. Its additional reductions come from more aggressive lane lending, metadata narrowing, lifetime splitting, and route-specific control simplification.

For nonzero $x \in \mathbb{F}_p$, the circuit implements

$$|x\rangle\,|y\rangle\,|0\rangle_W \longmapsto |x\rangle\,\left|yx^{-1} \bmod p\right\rangle\,|0\rangle_W\,, \tag{35}$$

where W is the Euclidean workspace. In Stage 2 of the low-Q point-addition circuit, $x = \widetilde{d}_x = x_A - x_R = -d_x$ and $y = \widetilde{d}_y = y_A - y_R = -d_y$, giving $\lambda = \widetilde{d}_y \widetilde{d}_x^{-1} = d_y d_x^{-1}$. Stage 5 uses the same machinery with the output-side witnesses $x = d'_x = x_A - x_{R'}$ and $y = d'_y = y_{R'} + y_A$, whose ratio is again λ.

Cross-cofactor invariant. The EEA begins with $\rho_0 = p$ and $\rho_1 = x$. At iteration i, it divides ρ_{i-1} by ρ_i and computes

$$\rho_{i-1} = q_i \rho_i + \rho_{i+1}, \qquad 0 \le \rho_{i+1} < \rho_i, \tag{36}$$

where q_i is the quotient and ρ_{i+1} is the remainder left by the division. The next iteration operates on (ρ_i, ρ_{i+1}). Thus, two consecutive remainders, meaning two adjacent values in the remainder sequence, must be stored simultaneously at each EEA boundary.

The extended algorithm also tracks how every remainder depends on the original inputs. Each remainder has the form $\rho_i = \beta_i p + \alpha_i x$ for integer coefficients α_i, β_i, and therefore $\rho_i \equiv \alpha_i x \pmod p$. If the algorithm reaches $\rho_m = 1$, then $\alpha_m x \equiv 1 \pmod p$, so $\alpha_m \equiv x^{-1} \pmod p$.

At a given boundary, denote the two simultaneously stored consecutive remainders by $\rho = \rho_i$ and $\rho' = \rho_{i+1}$, up to the ordering recorded by the role bit π, and denote their respective coefficients of x by $\alpha = \alpha_i$ and $\alpha' = \alpha_{i+1}$. Following the two-coefficient representation of Luo et al. [40], the packed implementation stores these coefficients as signed *cross-cofactors*

$$c = (-1)^{\pi}\alpha', \qquad c' = (-1)^{\pi+1}\alpha, \tag{37}$$

where $\pi \in \{0, 1\}$ records the current ordering of the two EEA states. They are crossed because c is derived from the coefficient associated with ρ', while c' is derived from the coefficient associated with ρ. Consecutive EEA states satisfy the determinant identity $\rho\alpha' - \rho'\alpha = (-1)^{\pi}p$. Substituting the definitions of c and c' gives the maintained invariants

$$\rho c + \rho' c' = p, \qquad \rho \equiv (-1)^{\pi+1} x c' \pmod p, \qquad \rho' \equiv (-1)^{\pi} x c \pmod p. \tag{38}$$

The initial state $(\rho, \rho', c, c', \pi) = (p, x, 1, 0, 0)$ satisfies these relations, and each paired remainder and cross-cofactor update preserves them throughout the EEA.

The representation makes each quotient update act on one remainder and the cross-cofactor associated with the opposite remainder. For a shifted quotient contribution $\kappa = 2^{\ell_s}$, exactly one of the following paired updates is applied:

$$\begin{aligned} \rho &\leftarrow \rho \mp \kappa\rho', & c' &\leftarrow c' \pm \kappa c, & &\text{when } \rho \text{ is the target,} \\ \rho' &\leftarrow \rho' \mp \kappa\rho, & c &\leftarrow c \pm \kappa c', & &\text{when } \rho' \text{ is the target.} \end{aligned} \tag{39}$$

The two lines are symmetric and are not performed simultaneously. In the implementation, one orientation is used during a Euclidean division; when that division finishes, the remainder/cofactor roles are exchanged and π is toggled.

The opposite signs preserve the central invariant. For example, the subtractive update in the first line gives $(\rho - \kappa\rho')c + \rho'(c' + \kappa c) = \rho c + \rho' c' = p$. The second line preserves the invariant by the same calculation.

Register sharing. Luo et al.'s construction stores the active EEA state in two shared qubit banks:

$$W_1 = c \parallel q \parallel \rho, \qquad W_2 = \mathrm{rot}_{\ell_s}(c' \parallel \rho')\,. \tag{40}$$

Here q is the partial quotient for the current division. The algorithm constructs $q_i = \lfloor \rho/\rho' \rfloor$ one binary digit at a time by testing shifted multiples $2^{\ell_s}\rho'$. Each accepted subtraction writes the corresponding quotient bit into q; after the completed quotient has controlled the associated cross-cofactor updates, its storage is cleared. The cursor ℓ_s records the circular offset of $(c' \| \rho')$ within W_2. Changing this offset aligns ρ' as $2^{\ell_s}\rho'$ and aligns the corresponding cross-cofactor operation without allocating separate shifted copies.

Initially, the remainders occupy most of the banks and the cross-cofactors and partial quotient are small. As the remainders shrink, the other fields expand into their vacated lanes. Scheduled length bounds identify the active field boundaries and keep each bank within $n + 3 = 259$ qubits for $n = 256$. This two-bank layout, including quotient packing, cursor-based alignment, length metadata, and baseline active-window arithmetic, comes from Luo et al. [40].

The challenge implementation integrates this layout into the complete mixed point-addition circuit and reduces its transient peak through more aggressive lifetime management. Temporarily inactive carry, comparison, coefficient, metadata, and coordinate lanes are loaned to local operations and restored before their owners become active again. It also narrows one length field, splits comparison and rotation lifetimes, fuses selected predicates and boundary operations, and specializes several guards to the validated route.

The implementation executes a fixed schedule of 1,479 reversible microsteps, using reversible padding when the EEA terminates early. At the normalized endpoint, $\rho = 1$, $\rho' = 0$, and $q = 0$. Equation (38) then gives $c' \equiv (-1)^{\pi+1}x^{-1} \pmod{p}$, so a sign correction recovers the inverse. After multiplying it into y, the circuit reverses the schedule to restore x and clear the work banks and metadata.

The two banks and principal metadata use 567 qubits. Together with the challenge-specific lifetime refinements, this architecture reduces the complete point-addition circuit from 947 to 851 logical qubits, while increasing average executed Toffoli count from 61,946,637 to 464,495,956. It is therefore a width-oriented trade-off. The EEA and cross-cofactor invariants are exact, whereas the submitted route's support-tuned active widths, truncated guards, and lane-hosting assumptions are validated only on the benchmark corpus and should not be interpreted as an all-input correctness result.

5.3.6 Constant Propagation The optimization referred to as *constant propagation* (*Constprop*) exploits control information that is classical for each evaluation instance rather than stored in quantum superposition. Let the classical multiplier be $y = \sum_{i=0}^{n-1} y_i 2^i$, where $y_i \in \{0, 1\}$. A quantum-by-classical schoolbook multiplication decomposes as

$$xy = \sum_{i=0}^{n-1} y_i (2^i x), \tag{41}$$

so each multiplier bit selects a modular update of an accumulator a:

$$a \longmapsto \begin{cases} a + 2^i x \pmod{p}, & y_i = 1, \\ a, & y_i = 0. \end{cases} \tag{42}$$

Because y_i is classically available, the corresponding arithmetic block can be executed only when $y_i = 1$. When $y_i = 0$, the block implements the identity and may therefore be omitted without changing the circuit's input–output behavior. This is an exact specialization, not an approximation.

The same transformation would not apply if y_i were held in quantum superposition, because the arithmetic operation would then require coherent quantum control. In the present setting, however, direct conditioning on the classical multiplier bits prevents inactive modular-addition and subtraction blocks from contributing to the executed-gate metric. This specialization reduced the multiplier's average executed Toffoli count from 55,277,050 to 54,495,466, a reduction of 781,584, while leaving the peak logical width unchanged. More generally, the expected saving scales with the frequency with which the relevant classical control bits are zero over the evaluation distribution.

5.3.7 Dead Code Elimination Dead-code and redundancy elimination removes operations, register capacity, and loop iterations that do not affect the required output. Self-inverse operations satisfying

$$f \circ f = \mathrm{id}$$

can be canceled, while consistent operand ordering and sign conventions eliminate unnecessary negations. Modular-reduction folds, carry propagation, and corrections can likewise be shortened or removed when their triggering conditions are provably unreachable. These exact transformations primarily reduce the executed Toffoli count.

Arithmetic and comparisons can also be restricted to their live bit ranges. The early and late iteration bounds

$$\begin{aligned} \max(r, s) &\le 2^{iter_idx}, \\ \mathrm{bitlen}(u) + \mathrm{bitlen}(v_w) &\le 2n - iter_idx. \end{aligned}$$

identify bits that must remain zero, permitting width-truncated arithmetic and removal of controls whose branches coincide. Although these transformations reduce local operation width, they affect peak logical width only when the modified operation lies on a width-determining phase.

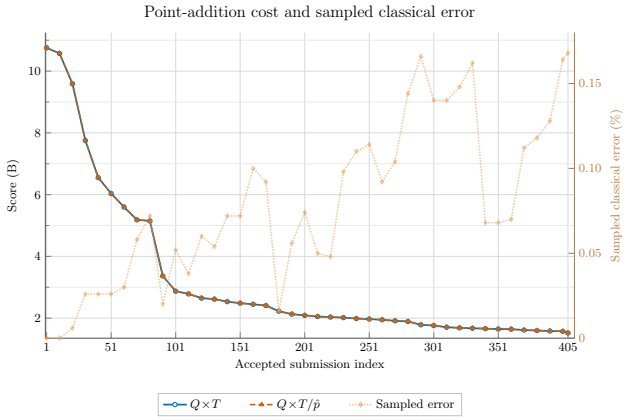
5.4 Evolution of the ECDSA.Fail Point-Addition Circuit

Over the course of the challenge, successive submissions reduced the point-addition score through a combination of major structural advances and sustained incremental refinement. In this section, we first trace the progression of the $Q \times T$ score over time (Section 5.4.1) and organize the major changes into three resource-model eras (Section 5.4.2). We then examine the separate efforts to construct sub-1,000-qubit circuits and to explore the broader Pareto frontier (Section 5.4.3). Finally, we classify the optimization techniques applied across the successive submissions, distinguishing frequently used incremental refinements from less common structural transformations associated with major frontier advances (Section 5.4.4).

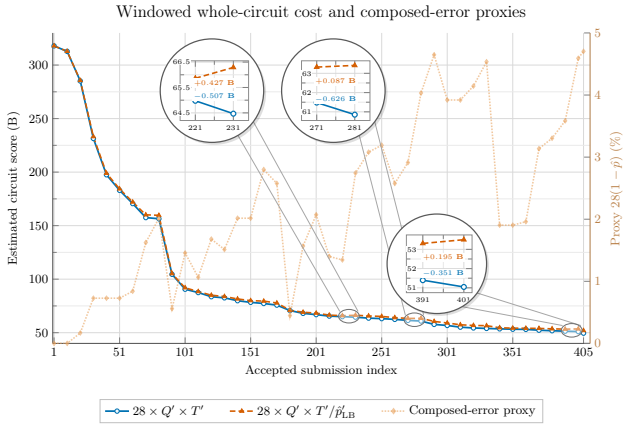
5.4.1 Raw and Retry-Adjusted $Q \times T$ Scores Across Submissions

Because many submissions use approximate arithmetic, a reduction in raw $Q \times T$ may be accompanied by a lower probability of producing the correct output. To determine whether the observed resource improvements persist after accounting for this trade-off, we re-evaluate a longitudinal sample of 42 accepted submissions on a common corpus of 50,000 pseudorandom inputs and estimate each circuit's empirical success probability $\hat{p}$. Under an independently rerunnable classical-success model, obtaining one successful point addition requires $1/\hat{p}$ executions on average, so $Q \times T/\hat{p}$ estimates the expected work per successful result. Writing the empirical error rate as $\hat{\epsilon} = 1 - \hat{p}$, the repetition factor is $1/\hat{p} = 1/(1-\hat{\epsilon}) \simeq 1 + \hat{\epsilon}$ for small $\hat{\epsilon}$, corresponding to approximately $100\hat{\epsilon}\%$ additional expected work. Figure 9 therefore compares the raw score $Q \times T$, the retry-adjusted proxy $Q \times T/\hat{p}$, and the observed classical-output error rate $1 - \hat{p}$ across the sampled submissions, ordered by their positions in the acceptance sequence.

As Figure 9 shows, the point-addition score decreases from approximately 10.7B to 1.5B across the sampled trajectory, while the corresponding raw proxy for a windowed full Shor circuit decreases from roughly 320B to 50B. The trajectory is distinctly stepwise, with major reductions followed by extended plateaus and smaller incremental gains. At the point-addition level, the retry-adjusted score $Q \times T/\hat{p}$ generally tracks the raw $Q \times T$ score because the observed classical-output error rates remain small. At the full-circuit level, the adjusted proxy assumes 28 point additions and estimates their aggregate success probability using the union bound $\hat{p}'_{\mathrm{LB}} = 1 - 28(1 - \hat{p})$; under this model, the highlighted intervals show that some nominal resource improvements diminish or reverse after accounting for the accumulation of sampled error across the complete computation. These patterns motivate analyzing the trajectory as a succession of circuit-level resource models rather than as a smooth sequence of scalar-score reductions.



(a) Point-addition retry proxy. The left axis compares $Q \times T$ with the plug-in quantity $Q \times T/\hat{p}$; the latter has an expected-work interpretation only under an independently rerunnable classical-success model. The right axis reports sampled classical-output error $100(1-\hat{p})$.



(b) Windowed whole-circuit proxy. The raw estimate is $28Q'T'$; the adjusted curve divides by the plug-in union-bound proxy $\hat{p}'_{\mathrm{LB}} = 1 - 28(1-\hat{p})$. The right axis shows the corresponding composed-error proxy $28(1-\hat{p})$.

Fig. 9: Resource-cost evolution in a later longitudinal sample of 42 accepted commits (indices $1, 11, \ldots, 401$, plus 405), extending through 20 July. Each commit is re-evaluated on the same fixed pseudorandom corpus of 50,000 elliptic-curve point pairs; hence $\hat{p}$ and T are sample estimates and can differ slightly from the official 9,024-instance operating points elsewhere in this section. Following (11), panel (b) uses the analytic sensitivity proxy $Q' = Q + 16$, $T' = T + 3 \cdot 2^{16}$, and 28 point additions at $n = 256$ and $w = 16$ (Section 2.2); it does not use the subsequently implemented windowed-addition-compatible circuit of Section 5.2.3, which directly measures the coherent-lookup overhead for the best-scoring circuit. The error composition is illustrative: it assumes the sampled classical-output error transfers to the windowed attack setting. It incorporates neither sampling uncertainty nor the separately recorded ancilla- and phase-garbage outcomes.

Era	Q trajectory	Inversion architecture	Principal architectural change
Era I	2,715 → 2,002	Two-pass Kaliski inversion	Packing, recomputation, and live-range reduction within the inherited architecture.
Era II	2,002 → 1,168	Streamed dialog-GCD inversion	Replacement of the history stack by a compressed, replayed Euclidean transcript.
Era III	1,167 → 1,151	Re-architected Jump-2 dialog-GCD	Off-peak addend materialization, base-5 coding, streamed replay, and coordinated local packing.

Table 4: Three architectural eras in the product-score trajectory.

The separation between the curves in Figure 9b is a sensitivity analysis, not a validated full-attack failure estimate. The plotted lower-bound proxy substitutes the empirical classical-output error for an applicable coherent per-addition error; it is neither a confidence bound nor evidence of coherent success. If it were applicable to an independently rerun computation, a 5% composed error would imply a repetition factor of $1/(1-0.05) \simeq 1.053$, or about 5.3% additional expected work, not a fivefold overhead. The derivation and its scope conditions appear in Supplemental Note A: Detailed results and circuit-improvement trajectory.

Finally, note that what we analyzed above is just a sample of the submissions. The pinned 27 July API response contains 831 rows, of which 826 were created by the data cutoff: 412 API-accepted, 253 API-rejected, and 161 API-failed entries.

The stepwise trajectory also reflects the collective organization of the search. Cross-referencing the frontier transitions with the public submission chronology shows that several plateau-ending advances came from participants other than those responsible for the preceding improvements, including both established and newly joined contributors. Once promoted, these advances became available for adoption and refinement by the wider community. The trajectory therefore records a sequence of public handoffs among partially independent search efforts, rather than the uninterrupted optimization path of a single participant or agent.

5.4.2 Three Eras of Circuit Optimization for the $Q \times T$ Score The promoted trajectory separates into three eras distinguished by their inversion architecture and dominant live-state constraint. Era I retained the two-pass Kaliski inverter and reduced its peak through scratch packing, recomputation, and shorter live ranges, reaching approximately 2,002 qubits without changing the underlying inversion method.

Era II adopted the record-and-replay Euclidean architecture introduced by Khattar et al. [33] and adapted to ECDLP point addition by Schrottenloher [59]. Replacing the persistent Kaliski history with a compressed, streamed transcript

reduced the width to 1,168 qubits, where the two coordinates of the classically fixed addend became a principal remaining peak owner.

Era III began with a TrailMix-derived reimplementation [64] built around a Jump-2 Euclidean recurrence. Its opening point of 1,167 qubits improved the width record by a single qubit, but it removed the 1,168-qubit wall: the fixed addend was now materialized only during off-peak coordinate operations. Combined with base-5 transcript encoding, streamed replay, specialized arithmetic, and tighter register lifetimes, this reorganization reached the 1,151-qubit incumbent at the data cutoff. The trajectory therefore resembles a verified ratchet: occasional architectural changes opened new resource regimes, after which numerous local refinements consolidated the gains. The individual mechanisms are described in Section 5.3, while detailed transitions and neighboring controls appear in Supplemental Note A.

5.4.3 Low-Qubit and Pareto Exploration The low-qubit track targets aggressive width reduction below 1,000 qubits by exchanging resident quantum state for recomputation, compact metadata, and extensive register reuse. An open low-width TrailMix route provided the principal implementation substrate [64], which challenge participants further integrated and specialized to obtain admitted operating points at $Q = 980$, 948, 828, 826, and 825. Along this branch, the work increase becomes abrupt, rising from 54.8M Toffoli gates at $Q = 948$ to 448.1M at $Q = 828$ and 489.2M at $Q = 825$. These endpoints demonstrate that the circuit can be compressed into a substantially lower-width regime and suggest a sharp width–work transition within the circuit families discovered so far. They do not, however, establish a fundamental hard-Q boundary: the product-efficient and low-width branches use different inversion architectures, so the sparsely populated region between them may reflect this architectural discontinuity, a steep generic time–space trade-off, or the absence of a hybrid or otherwise undiscovered construction that bridges the two regimes. Within the existing extreme-width routes, source reconstruction identifies quotient metadata, hosted predicates, and denominator-witness lifetimes as immediate constraints, without implying that these constraints are unavoidable or assigning a unique mechanism to every movement of the frontier.

A separate Pareto-frontier exploration produced a sequence of circuits spanning $Q = 1{,}150$ to $Q = 1{,}133$ while retaining average executed Toffoli counts near 1.4M. Representative points include $(Q, T) = (1,150,\ 1,284,776)$, $(1,141,\ 1,423,723)$, and $(1,133,\ 1,460,511)$. Although these circuits do not attain the minimum $Q \times T$ score, they reduce logical width with only a modest increase in nonlinear work; for example, the $Q = 1{,}133$ circuit uses approximately 12.4% more Toffoli gates than the $Q = 1{,}151$ incumbent while requiring 18 fewer logical qubits. Subject to the circuit-interface and full-Shor compatibility considerations discussed elsewhere, these Pareto points may be particularly relevant to Q-day resource scenarios in which logical-qubit capacity is more restrictive than execution time. The selected low-qubit and Pareto sequences, together

with the unsuccessful batching construction, are documented in Supplemental Note A: Detailed results and circuit-improvement trajectory.

5.4.4 Breakdown of Optimization Techniques Across the Circuit Optimization Trajectory Complementing the three-era chronology, Figure 10 classifies the 400 scored accepted source commits of the public competition through 18 July by their primary optimization family and, where applicable, a more specific change type. Throughout this classification we distinguish the *pre-competition development* history (commits #1–882), in which most of the techniques were first implemented, from the *public competition* that followed (accepted contestant commits #883–1294), which the census counts; this split is by provenance in the source history and is independent of the three resource-model eras of Section 5.4.2, which instead partition the promoted-submission trajectory by dominant width constraint. Supplemental Note B: List of agent-identified optimizations provides detailed descriptions for the circuit optimization techniques listed in the figure. Each assignment is based on an audit of the implemented source diff rather than the commit title or configuration-lever name, which frequently provides an incomplete or misleading description of the actual circuit change. To produce a mutually exclusive census, each commit is assigned a single primary family even when it combines several mechanisms; secondary changes therefore do not contribute additional counts. The figure excludes the four accepted but unscored bootstrap commits and the earlier pre-competition development history, so it characterizes the techniques used to refine the circuit during the public competition rather than establishing where a technique originated or who first proposed it. Submission `3616dbf` and its ping-pong architecture lie outside both the 18 July census interval and the 26 July evidence cutoff. They therefore do not alter the two algorithmic-substitution commits or any other counts in Figure 10.

Dead-code and redundancy elimination is the most frequent family, accounting for 244 of the 400 commits (61.0%). Most of these changes narrow comparators or arithmetic operations to the bits that can affect the result (149 commits), while iteration or threshold tuning and fold or carry truncation each account for 37 commits, and dead-gate or self-inverse cancellation accounts for 21. Register and scratch allocation is the second-largest family, with 50 commits (12.5%), including 30 ancilla-lifetime or venting changes, 17 scratch-reuse changes, and three in-place mutations. The remaining families are less frequent but span a broader range of structural mechanisms: measurement-based uncomputation appears as the primary change in 22 commits; constant propagation in 19; strength reduction in 14; fusion in 12; windowing or iteration blocking in eight; codec and transcript encoding in seven; and algorithmic substitution in two. Within these smaller families, the census identifies concrete recurring patterns, including boundary controlled-swap merging, fast-adder rerouting, windowed carry and shift blocks, Jump-2 transcript compression, and TrailMix/K5 codec improvements.

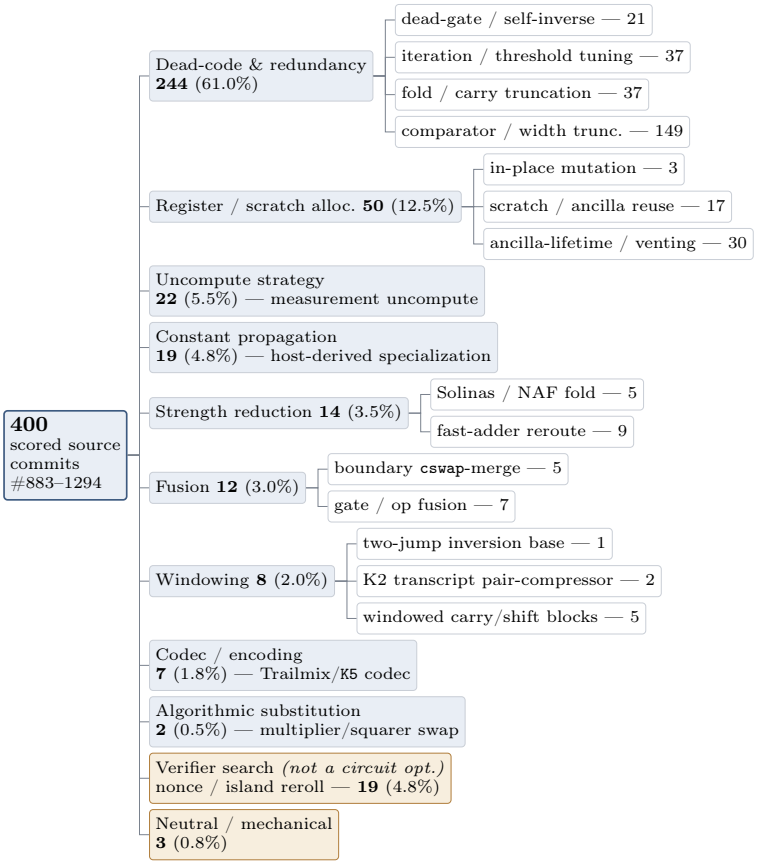


Fig. 10: Anatomy of the 400 scored, accepted source commits from the public competition through 18 July (#883–1294). Each commit is assigned to one primary optimization family (level 1) and, where applicable, one change type (level 2), based on an audit of its `git show` diff rather than its often-unreliable subject or lever name. Blue nodes represent nine circuit-optimization families: six conventional compiler categories and three reversibility-specific categories—uncompute strategy, register/scratch allocation, and *codec/encoding*—with distinct resource and correctness obligations. Amber nodes represent verifier search for a clean Fiat–Shamir nonce, which is not a circuit optimization, and neutral or mechanical changes. Counts measure how often each mechanism was the primary change, not its contribution to score reduction; accompanying nonce changes are attributed to the associated circuit optimization. Pre-competition development commits are excluded, including the representation/coordinate-change family documented only in earlier research (see Supplemental Note A: Detailed results and circuit-improvement trajectory), because no corresponding competition submission was accepted. Four accepted but unscored `DUMMY_TOFFOLIS` bootstrap commits are also excluded. The complete per-commit ledger, including those four commits, is provided in Supplemental Note C: List of accepted solutions.

Within the census interval, the distribution reflects how optimization proceeded after the then-current architectural components had stabilized. Most accepted commits performed localized refinement, particularly width truncation, threshold selection, and register-lifetime adjustment, whereas the larger structural transformations occurred much less frequently. Frequency, however, should not be interpreted as resource impact: the figure measures how often a technique was the primary change, not how much it reduced Q, T, or $Q \times T$. Several rare structural transformations, including Jump-2 inversion, fusion, and measurement-based uncomputation, are associated with disproportionately large movements of the frontier, while many high-frequency changes contributed smaller incremental gains. This distinction reinforces the three-era interpretation: a small number of architectural transitions created new resource regimes, after which numerous local optimizations exploited the resulting headroom.

Two residual categories are shown separately from the circuit-optimization taxonomy. Verifier search accounts for 19 commits (4.8%) whose primary change is a nonce or validation-island reroll rather than a modification of the semantic circuit, while three neutral or mechanical commits (0.8%) do not materially change the emitted circuit. When a commit combines a genuine circuit optimization with a nonce reroll, it is classified under the circuit transformation; verifier search is used only when no substantive circuit change is identified. The complete commit-level classifications, score joins, and attribution criteria are provided in Supplemental Note A: Detailed results and circuit-improvement trajectory and Supplemental Note C: List of accepted solutions.

5.5 Reproducibility of the Results

We support two complementary forms of reproducibility: replaying the reported circuit artifacts and reconstructing the frontier analysis from frozen public data. Circuit replay verifies the correctness and measured resources of a selected implementation, whereas analysis reproduction verifies the submission selection, Pareto construction, tables, and figures.

Circuit reproduction. The companion manifest links each reported Pareto point to a submission identifier and an immutable source commit, so any point can be reproduced offline from the public repository, without the leaderboard service. After cloning the benchmark once,

```
git clone https://github.com/Layr-Labs/ecdsafail-challenge.git
cd ecdsafail-challenge
```

each operating point is rebuilt by checking out its source commit and rerunning the pinned evaluator. For example, the best $Q \times T$ circuit `8e9c9a2` (the first promoted submission crossing the 50% score-reduction threshold) and the post-cutoff ping-pong circuit `3616dbf` are reproduced by:

```
# best Q x T circuit (8e9c9a2, score 1,495,670,403)
git checkout 60d61859fa6965eba53634f91877b1141a6f9dce
```

```
./setup.sh && ./benchmark.sh

# post-cutoff ping-pong circuit (3616dbf, score 1,258,525,947)
git checkout 897dda2b0cf267151ecd973252d2a5078cbf1b63
./setup.sh && ./benchmark.sh
```

Each checkout restores the complete historical repository state for that submission, including its circuit source, the evaluator, and the pinned toolchain; because the manifest pins every Pareto point to such a commit, each reported point has an immutable, self-contained reproduction. The evaluator attempts 9,024 Fiat–Shamir-derived draws, skips degenerate pairs, and reports the number actually tested as `n_shots`. For each retained input, it executes the submitted operation stream once in the forward direction, checks classical outputs, phase cleanliness, and final ancilla cleanup, and reports Q, T, and $Q \times T$ in `score.json`.

Frozen data and frontier reconstruction. The quantitative analyses use an official API response fetched on 27 July 2026 and a data cutoff of 26 July 2026 at 09:21:55 UTC, defined by the first promoted submission achieving a 50% score reduction relative to the Google/Babbush low-gate reference. The frozen response contains 831 rows, including 826 created by the cutoff and 576 with structured Q and T metrics. It is joined with the 80-row contribution chronology used in Section 5.4, which supplies the selected milestones and mechanism labels. The historical display contains 15 manifest-selected transitions and is an editorial reconstruction rather than a statistical sample. The frozen public data, selection manifest, reconstruction code, tests, and generated outputs are available in the frontier reconstruction package.

6 Limitations, Safety, and Responsible Framing

The results of Section 5 are operating points on a public competition leaderboard, not validated fault-tolerant resource estimates. Their interpretation therefore requires attention to measurement integrity, finite-support correctness, and attribution within human–agent workflows.

The score is sampled and can be optimized through support selection. The benchmark measures average *executed* Toffoli count over a deterministic, submission-dependent set of Fiat–Shamir inputs (Section 3.3). Because the input set is derived by hashing the submitted operation stream, changing a submission's nonce can select a more favorable validation "island" on which the same semantic circuit executes fewer Toffoli gates. Such nonce search is not a circuit optimization, yet it can lower the reported score. It constitutes the *verifier-search* bucket of Figure 10 (4.8% of scored source commits), and approximately half of the audited commits include a reroll alongside a circuit modification. We distinguish rerolling from gate removal by comparing source changes and emitted operation streams, including static Toffoli counts after excluding nonce-tail and padding changes. An improvement is classified as verifier search only when these comparisons identify no semantic circuit-body change.

Approximate circuits can pass finite validation. A transformation that fails on a small fraction of inputs may still pass the 9,024-instance verifier if its nonce is selected to avoid the failing cases. Such a circuit is validated only over the selected support and does not establish all-input correctness. We therefore classify approximate reductions by their circuit mechanism while explicitly reporting their support-conditioned status. The archive also contains `DUMMY_TOFFOLIS`, a self-canceling padding mechanism that changes the reported count without altering the circuit's semantics. These cases demonstrate that the leaderboard score is an empirical cost over finite, submission-selected support rather than a certified bound on correctness or cost over the complete input domain.

Human insight and agent automation are complementary. AI agents proved effective at implementing, testing, and refining circuit optimizations, particularly across the many incremental changes that would otherwise require substantial manual effort. They also occasionally helped develop structural transformations. The post-cutoff comparison-free ping-pong dialog-GCD provides the clearest example: an agent-mediated workflow combined prior plus-minus GCD and dialog-replay ideas into a structurally distinct fixed-alternation backend that substantially reduced the Toffoli count (Section 5.3.3). At the same time, human research remained essential. The earlier dialog-GCD improvement followed the record-and-replay architecture introduced by Khattar et al. [33] and adapted to ECDLP point addition by Schrottenloher [59], while humans supplied research connections, objectives, interpretation, and technical review. Accordingly, *agent-identified* denotes an optimization surfaced, adapted, combined, or developed within an agent-mediated workflow; it does not imply autonomous invention, exclusive model authorship, or independent novelty.

Windowed point-addition compatibility. The benchmark optimizes mixed addition with a classically known addend, whereas windowed Shor (Section 2.2) selects the addend coherently through a QROM lookup (10), so optimizations that specialize on classical coordinates must be replaced by coherent operations. We implemented a windowed-compatible variant of the best-scoring circuit (Section 5.2.3) that retains the arithmetic core within a lookup/use/unlookup construction, at an overhead of 11 logical qubits and about 29.6% average executed Toffoli per addition and no detectable error-rate difference on the paired corpus (29 windowed-only versus 30 baseline-only failures; two-sided exact sign test, $p = 1.0$). This demonstrates the adaptation at the point-addition level but covers only a single call on the generic-affine domain of Section 3.1, which for nonidentity A excludes $R \in \{A, -A, -2A\}$; coherent correctness is argued from linearity rather than established through direct simulation. We therefore reserve full-Shor claims until the complete coherent construction is assembled and evaluated end to end.

The public archive has an incomplete attempt denominator. The accepted-commit history records solutions that reached the public branch, and the submissions API records candidates that were sent to the service. It does not

exhaustively record local experiments, discarded branches, failed builds, private agent sessions, or hypotheses that participants chose not to submit. Even the population of rejected submissions is therefore a selected subset of all attempted work. The optimization census and chronology characterize observable artifacts, not the number or distribution of research attempts. They cannot by themselves estimate success rates per attempt, discovery time, the frequency of negative results, or the productivity of a participant or tool. Claims about failure accumulation are limited to failures that were actually preserved in a public note, submission, commit, or deposited session.

The trajectory is observational rather than a causal comparison. Participants self-selected their tools, starting commits, objectives, compute budgets, and the results they exposed. Tool adoption also changed over time as the circuit, community, and available base solutions evolved. There was no random assignment to human-only and agent-assisted workflows, no common attempt budget, and no complete prompt or wall-clock log. The observed record supports the descriptive claim that substantial improvements occurred in an agent-mediated open workflow. It does not isolate the counterfactual improvement that the same participants would have achieved without agents, nor establish that one model, prompting strategy, or collaboration structure caused a given frontier movement. We therefore keep implementation provenance, contributor statements, and causal effectiveness claims distinct.

Responsible framing. This work improves a reversible point-addition primitive relevant to quantum attacks on elliptic-curve cryptography, but it does not constitute an executable attack or a complete fault-tolerant resource estimate. The reported circuits omit physical error correction, architecture-dependent compilation, full non-Clifford accounting, and validated integration into windowed Shor arithmetic. The results therefore refine the understanding of one logical-resource bottleneck rather than establish an immediate capability to break `secp256k1`. We report them openly to support reproducible research, informed cryptographic risk assessment, and the timely transition to post-quantum cryptography.

7 Conclusion and Outlook

This work presented ECDSA.Fail, an open effort to optimize reversible secp256k1 point-addition circuits relevant to Shor's algorithm for the elliptic-curve discrete logarithm problem. Starting from a baseline of 2,715 logical qubits and 3,960,753 average executed Toffoli gates, the challenge reduced the best $Q \times T$ operating point to 1,151 qubits and 1,299,453 average executed Toffoli gates, corresponding to an 86.1% reduction in the product metric. A separate low-width search reached 825 logical qubits at the data cutoff, subsequently improved to 813, exposing a markedly different region of the reversible time–space trade-off.

Beyond the resulting circuits, ECDSA.Fail provides a case study of *Open Autoresearch*: a verifier-gated research process in which human participants and AI

agents iteratively generate, implement, test, and share candidate improvements against a common measurable objective. The public repository, evaluator, leaderboard, and submission history allowed individually produced improvements to become common starting points for subsequent work, while the resulting source history made it possible to reconstruct both incremental refinements and larger architectural transitions. The experience suggests that human judgment, agent-scale experimentation, and open sharing can be complementary when progress can be evaluated rapidly and reproducibly, while the observational nature of the challenge does not by itself establish the causal contribution of any particular agent, model, or workflow.

The reported operating points should nevertheless be interpreted within the benchmark that produced them. Correctness is established over finite, submission-dependent validation support rather than over the complete input domain, and the scored primitive assumes a classically specified addend. We showed this last restriction to be an interface convenience rather than a fundamental barrier, building a coherent windowed-compatible variant of the best-scoring circuit at modest overhead (Section 5.2.3); this demonstration is limited to a single point-addition call, however, and the scored results do not by themselves constitute complete fault-tolerant attack estimates.

Building on the windowed-compatible point-addition kernel demonstrated here, the most important next step is to compose it into a complete windowed implementation of Shor's algorithm, incorporating all shifted tables, the full window schedule, and the Fourier and postprocessing layers, validated end to end and evaluated under resource-accounting conventions that permit direct comparison with prior work. Beyond secp256k1, the broader question is whether the open-autoresearch pattern observed here transfers to other hard optimization problems with machine-checkable objectives. ECDSA.Fail provides one concrete instance in which a public, human–agent research process produced substantial and source-auditable progress. Testing the generality of that model is a natural direction for future work.

Acknowledgments

We thank Craig Gidney and Tanuj Khattar of Google Quantum AI for carefully reviewing earlier drafts of this manuscript and for their constructive comments and suggestions, which substantially improved its technical accuracy, clarity, and presentation. Any remaining errors are our own.

References

1. Aharonov, D.: A simple proof that toffoli and hadamard are quantum universal. arXiv preprint quant-ph/0301040 (2003). https://doi.org/10.48550/arXiv.quant-ph/0301040
2. Aygün, E., et al.: An AI system to help scientists write expert-level empirical software. Nature **654**, 909–916 (2026). https://doi.org/10.1038/s41586-026-10658-6

3. Babbush, R., Zalcman, A., Gidney, C., Broughton, M., Khattar, T., Neven, H., Bergamaschi, T., Drake, J., Boneh, D.: Securing elliptic curve cryptocurrencies against quantum vulnerabilities: Resource estimates and mitigations (2026), arXiv:2603.28846, https://arxiv.org/abs/2603.28846
4. Bennett, C.H.: Logical reversibility of computation. IBM Journal of Research and Development **17**(6), 525–532 (1973)
5. Bernstein, D.J., Yang, B.Y.: Fast constant-time GCD computation and modular inversion. IACR Transactions on Cryptographic Hardware and Embedded Systems **2019**(3), 340–398 (2019), cryptology ePrint Archive, Report 2019/266, https://eprint.iacr.org/2019/266
6. biggiemac42: Official record submission thread. r/opus_magnum, Reddit (Aug 2020), https://www.reddit.com/r/opus_magnum/comments/i411ul/official_record_submission_thread/, community submissions feed the Opus Magnum leaderboard; accessed 16 August 2026
7. Bigou, K., Tisserand, A.: Improving modular inversion in RNS using the plus-minus method. Cryptology ePrint Archive, Paper 2015/193 (2015), https://eprint.iacr.org/2015/193
8. Bravyi, S., Kitaev, A.: Universal quantum computation with ideal clifford gates and noisy ancillas. Physical Review A **71**, 022316 (2005)
9. Cain, M., Xu, Q., King, R., Picard, L.R.B., Levine, H., Endres, M., Preskill, J., Huang, H.Y., Bluvstein, D.: Shor's algorithm is possible with as few as 10,000 reconfigurable atomic qubits (2026), https://arxiv.org/abs/2603.28627
10. Certicom Research: SEC 2: Recommended elliptic curve domain parameters, version 2.0. Tech. rep., Standards for Efficient Cryptography Group (2010)
11. Chia, N.H., Chou, C.N., Zhang, J., Zhang, R.: Quantum meets the minimum circuit size problem (2021), https://arxiv.org/abs/2108.03171
12. Cooper, S., Khatib, F., Treuille, A., Barbero, J., Lee, J., Beenen, M., Leaver-Fay, A., Baker, D., Popović, Z., Foldit Players: Predicting protein structures with a multiplayer online game. Nature **466**, 756–760 (2010)
13. Cuccaro, S.A., Draper, T.G., Kutin, S.A., Moulton, D.P.: A new quantum ripple-carry addition circuit (2004), arXiv:quant-ph/0410184, https://arxiv.org/abs/quant-ph/0410184
14. Dallaire-Demers, P.L., Doyle, W., Foo, T.: Brace for impact: ECDLP challenges for quantum cryptanalysis (2026), https://arxiv.org/abs/2508.14011
15. DeepSeek-AI: DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning (2025), arXiv:2501.12948, https://arxiv.org/abs/2501.12948
16. Ding, K., Yu, J., Huang, J., Yang, Y., Zhang, Q., Chen, H.: SciToolAgent: A knowledge-graph-driven scientific agent for multitool integration. Nature Computational Science **5**, 962–972 (2025). https://doi.org/10.1038/s43588-025-00849-y
17. ECDSA.Fail: ECDSA.Fail. https://www.ecdsa.fail/ (2026), accessed: 2026-07-20
18. ECDSA.Fail: ecdsafail-challenge. https://github.com/Layr-Labs/ecdsafail-challenge (2026), accessed: 2026-07-20
19. Fawzi, A., et al.: Discovering faster matrix multiplication algorithms with reinforcement learning. Nature **610**, 47–53 (2022)
20. Fösel, T., Niu, M.Y., Marquardt, F., Li, L.: Quantum circuit optimization with deep reinforcement learning (2021), arXiv:2103.07585, https://arxiv.org/abs/2103.07585
21. Franzoni, C., Sauermann, H.: Crowd science: The organization of scientific research in open collaborative projects. Research Policy **43**(1), 1–20 (2014)

22. Garn, M., Kan, A.: Quantum resource estimates for computing binary elliptic curve discrete logarithms (2025), arXiv:2503.02984, https://arxiv.org/abs/2503.02984
23. Gidney, C.: Halving the cost of quantum addition. Quantum **2**, 74 (2018)
24. Gidney, C.: Windowed quantum arithmetic (2019), arXiv:1905.07682, https://arxiv.org/abs/1905.07682
25. Gouzien, É., Ruiz, D., Le Régent, F.M., Guillaud, J., Sangouard, N.: Performance analysis of a repetition cat code architecture: Computing 256-bit elliptic curve logarithm in 9 hours with 126,133 cat qubits. Physical Review Letters **131**, 040602 (2023), https://arxiv.org/abs/2302.06639
26. Gowers, T., Nielsen, M.: Massively collaborative mathematics. Nature **461**, 879–881 (2009)
27. Häner, T., Jaques, S., Naehrig, M., Roetteler, M., Soeken, M.: Improved quantum circuits for elliptic curve discrete logarithms. In: Post-Quantum Cryptography – PQCrypto 2020. LNCS, vol. 12100, pp. 425–444. Springer (2020), https://arxiv.org/abs/2001.09580
28. Johnson, D., Menezes, A., Vanstone, S.: The elliptic curve digital signature algorithm (ECDSA). International Journal of Information Security **1**(1), 36–63 (2001)
29. Jones, C.: Low-overhead constructions for the fault-tolerant toffoli gate. Physical Review A **87**, 022328 (2013)
30. Kauers, M., Moosbauer, J.: Flip graphs for matrix multiplication. In: ISSAC 2023 (2023), arXiv:2212.01175
31. Kaye, P., Zalka, C.: Optimized quantum implementation of elliptic curve arithmetic over binary fields. Quantum Information & Computation **5**(6), 474–491 (2005). https://doi.org/10.26421/QIC5.6-6, https://arxiv.org/abs/quant-ph/0407095
32. Khattar, T., Gidney, C.: Rise of conditionally clean ancillae for efficient quantum circuit constructions. Quantum **9**, 1752 (2025), arXiv:2407.17966
33. Khattar, T., Shutty, N., Gidney, C., Zalcman, A., Yosri, N., Maslov, D., Babbush, R., Jordan, S.P.: Verifiable quantum advantage via optimized DQI circuits (2025), arXiv:2510.10967, https://arxiv.org/abs/2510.10967
34. Kjelstrøm, A.H., Pavlogiannis, A., van de Pol, J.: Exact quantum circuit optimization is co-NQP-hard (2026), https://arxiv.org/abs/2510.16420
35. Kornerup, N., Sadun, J., Soloveichik, D.: Tight bounds on the spooky pebble game: Recycling qubits with measurements. Quantum **9**, 1636 (2025), arXiv:2110.08973
36. Litinski, D.: How to compute a 256-bit elliptic curve private key with only 50 million toffoli gates (2023), arXiv:2306.08585, https://arxiv.org/abs/2306.08585
37. Long, J.: Large language model guided tree-of-thought (2023). https://doi.org/10.48550/arXiv.2305.08291, https://arxiv.org/abs/2305.08291
38. Lu, C., Lu, C., Lange, R.T., Foerster, J., Clune, J., Ha, D.: The AI scientist: Towards fully automated open-ended scientific discovery (2024), arXiv:2408.06292, https://arxiv.org/abs/2408.06292
39. Luo, H., Yang, Z., Luo, J., Wang, Z., Su, Y., Sun, X., Li, L., Li, T.: Quantum algorithm for elliptic curve discrete logarithms with space-efficient point addition (2026). https://doi.org/10.48550/arXiv.2607.13816, https://arxiv.org/abs/2607.13816
40. Luo, H., Yang, Z., Wang, Z., Su, Y., Li, T.: Space-efficient quantum algorithm for elliptic curve discrete logarithms with resource estimation (2026), arXiv:2604.02311, https://arxiv.org/abs/2604.02311
41. Mankowitz, D.J., et al.: Faster sorting algorithms discovered using deep reinforcement learning. Nature **618**, 257–263 (2023)

42. Massalin, H.: Superoptimizer: A look at the smallest program. In: ASPLOS 1987. pp. 122–126 (1987)
43. Nakamoto, S.: Bitcoin: A peer-to-peer electronic cash system. https://bitcoin.org/bitcoin.pdf (2008)
44. National Institute of Standards and Technology: Post-quantum cryptography standards: FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), and FIPS 205 (SLH-DSA). Federal Information Processing Standards Publications (2024), https://csrc.nist.gov/projects/post-quantum-cryptography
45. National Institute of Standards and Technology: Transition to post-quantum cryptography standards. NIST Internal Report 8547 (Initial Public Draft), National Institute of Standards and Technology (2024), https://csrc.nist.gov/pubs/ir/8547/ipd
46. Neri, C.: Shorter and faster than Sort3AlphaDev. arXiv:2307.14503 (2023), https://arxiv.org/abs/2307.14503
47. Nielsen, M.A., Chuang, I.L.: Quantum Computation and Quantum Information. Cambridge University Press, 10th anniversary edn. (2010)
48. Novikov, A., et al.: AlphaEvolve: A coding agent for scientific and algorithmic discovery (2025), arXiv:2506.13131, https://arxiv.org/abs/2506.13131
49. Office of Management and Budget: Execution of the migration to post-quantum cryptography. Memorandum M-26-15, Executive Office of the President of the United States (Jun 2026), https://www.whitehouse.gov/wp-content/uploads/2026/06/M-26-15-Execution-of-the-Migration-to-Post-Quantum-Cryptography.pdf, june 24, 2026
50. Parent, A., Roetteler, M., Mosca, M.: Improved reversible and quantum circuits for Karatsuba-based integer multiplication. arXiv:1706.03419 (2017), https://arxiv.org/abs/1706.03419
51. President's Working Group on Digital Asset Markets: Strengthening american leadership in digital financial technology. Tech. rep., Executive Office of the President of the United States (Jul 2025), https://www.whitehouse.gov/wp-content/uploads/2025/07/Digital-Assets-Report-EO14178.pdf
52. Proos, J., Zalka, C.: Shor's discrete logarithm quantum algorithm for elliptic curves. Quantum Information & Computation **3**(4), 317–344 (2003), https://arxiv.org/abs/quant-ph/0301141
53. Quist, A.J., Laarman, A.: Trade-offs between classical and quantum space using spooky pebbling. Logical Methods in Computer Science **21**(4) (2025). https://doi.org/10.46298/lmcs-21(4:29)2025, https://arxiv.org/abs/2401.10579
54. Roetteler, M., Naehrig, M., Svore, K.M., Lauter, K.: Quantum resource estimates for computing elliptic curve discrete logarithms. In: Advances in Cryptology – ASIACRYPT 2017. LNCS, vol. 10625, pp. 241–270. Springer (2017), https://arxiv.org/abs/1706.06752
55. Romera-Paredes, B., et al.: Mathematical discoveries from program search with large language models. Nature **625**, 468–475 (2024)
56. Ruiz, F.J.R., et al.: Quantum circuit optimization with AlphaTensor. Nature Machine Intelligence (2025), arXiv:2402.14396, https://arxiv.org/abs/2402.14396
57. Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., Scialom, T.: Toolformer: Language models can teach themselves to use tools. In: Advances in Neural Information Processing Systems (NeurIPS) (2023), arXiv:2302.04761

58. Schkufza, E., Sharma, R., Aiken, A.: Stochastic superoptimization. In: ASPLOS 2013. pp. 305–316 (2013)
59. Schrottenloher, A.: Optimized point addition circuits for elliptic curve discrete logarithms (2026), arXiv:2606.02235, https://arxiv.org/abs/2606.02235
60. Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., Yao, S.: Reflexion: Language agents with verbal reinforcement learning. In: Advances in Neural Information Processing Systems (NeurIPS) (2023), arXiv:2303.11366
61. Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. SIAM Journal on Computing **26**(5), 1484–1509 (1997)
62. Silverman, J.: The Arithmetic of Elliptic Curves. Graduate Texts in Mathematics, Springer New York (2009), https://books.google.com.tr/books?id=Z90CA_EUCCkC
63. Terwiesch, C., Xu, Y.: Innovation contests, open innovation, and multiagent problem solving. Management Science **54**(9), 1529–1543 (2008)
64. Trail of Bits: Trailmix (2026), https://github.com/trailofbits/trailmix
65. van de Wetering, J., Amy, M.: Optimising quantum circuits is generally hard (2024), https://arxiv.org/abs/2310.05958
66. Wood, G.: Ethereum: A secure decentralised generalised transaction ledger. Ethereum Project Yellow Paper, https://ethereum.github.io/yellowpaper/paper.pdf (2014), specifies the secp256k1 curve for transaction signatures
67. Yang, J., Jimenez, C.E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., Press, O.: SWE-agent: Agent-computer interfaces enable automated software engineering. In: Advances in Neural Information Processing Systems (NeurIPS) (2024), arXiv:2405.15793
68. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: ReAct: Synergizing reasoning and acting in language models. In: International Conference on Learning Representations (ICLR) (2023), arXiv:2210.03629

A Contributor Statement

The categories below are nonexclusive. Individual circuit submissions and their provenance remain recorded on the public leaderboard.

Paper contributors. Jieyi Long led the drafting, synthesis, coordination, and manuscript-wide integration of the paper. Theodore Pender, Zhao Huang, Manuel B. Santos, and Samrendra Kumar Singh contributed major technical and empirical sections, including the quantum-circuit background, circuit-optimization analysis, results, improvement chronology, and Pareto-frontier analysis. Justin Drake, Pierre-Luc Dallaire-Demers, and Bartosz Naskręcki contributed technical review and revision, and Francesco Giannicola contributed detailed review, reproducibility study, and editorial refinement.

Challenge organizers. Gajesh Naik, Gautham Anant, Soubhik Deb, and Justin Drake contributed to the conception, harness setup, organization, infrastructure, administration, or supervision of the challenge.

Leaderboard contributors. The challenge's progress reflects the cumulative contributions of participants who performed circuit research, implementation, testing, validation, and submission work. Within this broader collective effort, several key optimization techniques and plateau-breaking improvements were identified by Theodore Pender, BitWonka, Bartosz Naskręcki, Joe Doyle, Matt Zweil, and Gajesh Naik, working in collaboration with their AI agents. These particularly consequential contributions complemented the many other advances, refinements, and validation efforts made by the wider leaderboard community. **Named leaderboard contributors**: Akash Balasubramani, BitWonka, John Boyer, Xavier Butler, Pierre-Luc Dallaire-Demers, Bereket Dereje, Michael Dong, Joe Doyle, Oli Freuler, Vasily Gnuchev, Alexander Hertlein, Zhao Huang, Anto Joseph, Robert Kodra, Edison Lee, Jackie Chia-Hsun Lee, Lucas Levy, Alan Li, Jieyi Long, Gajesh Naik, Bartosz Naskręcki, Jordan Newman, Ruben Marcus Luz Paschoarelli, Shaan Patel, Theodore Pender, JT Rose, Manuel B. Santos, Samrendra Kumar Singh, Okechukwu Wisdom. **Other Leaderboard Contributors (Github ID)**: 0xfoobar, 0xnirlin, 0xpg, 10d9e, abipalli, aburan28, agilexmarketing, alecstecpe-oss, AnalyticETH, Angel-Arevalo, anshu4321, anupsv, austinamissah, ayushgupta0610, blocksek, borelien, bulengerk, bxue-l2, chainyoda, Chris-Moller, cryptogakusei, darius-oai, DavetheSlayer, davidxia3, dennisonbertram, eferbarn, eiso, factory-sagar, fortunexbt, Gaijin-01, gopikannappan, harrigan, hydrogenbond007, inishc, IntrepidEnki, jacksonhblau, jamesotter99, Jamgiter, Joshgriste, josusanmartin, jtaroreh, junpenglao, kamilsa, kunalvg, lucasabu1988, Luigy-Lemon, Lulu-Zhou-EigenLabs, M3kko, makimakiver, megabyte0x, mmurrs, mochimodev, mpjunior92, nandy-technologies, nikhiljha, nnn-gif, novoyd, pavvann, phileigenlabs, philuponcrypto, pq-cybarg, prasiddhnaik, pschork, ptrdsh, runeape-sats, rwnq8, Sam-Scolari, Sampriv, saucegodbased, simonik11, skb93, solimander, stevenhao, steventhornton, swadhin, Tasfia-17, tekkac, TeS44, tmid1, twangodev, twmmason, vatsalkshah, vineetguptadev, wh1sky02, xadenryan, ygboucherk, YQ-Wang, yudduy, yudongcao, zeeshan8281, zigtur, zpano, zuiris.